bezierkit 套件：Bézier 曲線輸出工具

Pin Yue Sung[*†]　　Ling Tak Douglas Chung[*]

2026-10-04 · v1.0.0 · ·

目錄

1 簡介 2
1.1 符號 2
1.2 數學與證明 2
1.3 閱讀指引 2
2 安裝 3
2.1 系統需求 3
2.2 安裝套件 3
2.3 開發環境設定 3
3 快速開始 4
3.1 一條三次曲線 4
3.2 主要元件 4
4 幾何數值物件 4
4.1 點與向量 5
4.2 點集合 5
4.3 參數 5
4.4 例外 5
5 Bézier 曲線 6
5.1 Bernstein 基底 6
5.2 曲線 7
5.3 求值 8
5.4 升階 9
6 導數、反轉與分割 9
6.1 導數 9
6.2 反轉 10
6.3 分割 10
7 三次線段與路徑 11
7.1 三次線段 11
7.2 緊密邊界框 12
7.3 升階 12
7.4 分段路徑 13
7.5 連續性 14
8 建構與 Hermite 插值 15
8.1 端點建構 15
8.2 Hermite 插值 16
8.3 插值誤差 17
9 擬合 18
9.1 自適應 Hermite 擬合 18
9.2 折線簡化 20
10 等值線 21
10.1 等值集與切線 21
10.2 描繪 22
10.3 結果 24
10.4 限制 24
11 取樣與匯出 24
11.1 取樣 24
11.2 四捨五入與座標變換 24
11.3 JSON 25
11.4 SVG 25
11.5 TikZ 26
11.6 Matplotlib 26
12 命令列介面 26
12.1 控制點 27
12.2 指令 27
12.3 錯誤 27
A 證明 27
A.1 Bernstein 基底與求值 28
A.2 導數、反轉與分割 30
A.3 三次線段與路徑 31
A.4 建構與插值 33
A.5 擬合與等值線 34
A.6 匯出 36
更新紀錄 37
指令索引 38
參考文獻 39

[*]國立政治大學國際經營與貿易學系，臺北，臺灣。
[†]通訊作者。電子郵件：contact@econ-viz.org

第 1 節　簡介

bezierkit 是處理 Bézier 曲線的小型數學工具套件。它可由控制點或端點條件建構曲線，計算曲線值與導數，分割與截取曲線，將曲線擬合到函數、取樣點與等值線，並輸出為 JSON、SVG 路徑資料或 TikZ。套件本身不繪圖。Matplotlib、mosaickit 或 LaTeX 文件等繪圖端接收精確的三次控制點，再負責繪製。

1.1　符號

點或向量位於某個維度 $d \ge 1$ 的 $\mathbb{R}^d$ 中；多數圖使用 $d = 2$。$x \in \mathbb{R}^d$ 的歐氏範數為

$$\|x\| = \sqrt{x_1^2 + \dots + x_d^2},$$

$\|x\|_\infty = \max_k |x_k|$ 為最大範數。集合 $S \subseteq \mathbb{R}^d$ 若對所有 $p, q \in S$ 與 $\lambda \in [0,1]$ 都有 $\lambda p + (1-\lambda) q \in S$，稱為凸集；有限個點的凸包是它們所有凸組合 $\sum_i \lambda_i P_i$（$\lambda_i \ge 0$ 且 $\sum_i \lambda_i = 1$）的集合。映射 $A : \mathbb{R}^d \to \mathbb{R}^e$ 若可寫成 $A(x) = Mx + v$（M 為矩陣，v 為向量），稱為仿射映射。函數若有直到 k 階的連續導數，稱為 C^k。

n 次 Bézier 曲線有 $n+1$ 個控制點 $P_0, \dots, P_n$，定義為映射

$$B(t) = \sum_{i=0}^{n} b_{i,n}(t) P_i, \quad t \in [0,1],$$

其中

$$b_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}$$

為 Bernstein 多項式（詳見第 5 節）。依序連接控制點即為控制多邊形。套件中每條曲線的參數範圍都是 $[0,1]$；超出範圍的參數會拋出 `ParameterOutOfDomain`。符號 d 一律表示維度，容許誤差寫作 ε。

1.2　數學與證明

各章先回顧所用的標準定義並附出處，再以編號的引理、命題、定理與推論陳述演算法所依據的性質：Bernstein 基底、de Casteljau 演算法的求值與分割、Hermite 誤差界，以及匯出器的捨入誤差。證明集中在附錄 A，只想了解 API 時可略過。屬於本套件而不見於文獻的約定，稱為套件約定。標準參考書為 Farin (2002) 與 Prautzsch et al. (2002)；Bernstein 基底另見 Farouki (2012)。

1.3　閱讀指引

表 1　章節主題

主題	內容	章節
基礎	點、向量、參數、例外	第 4 節
	Bernstein 基底、曲線、求值	第 5 節
	導數、反轉、分割	第 6 節

主題	內容	章節
三次曲線	三次線段與分段路徑	第 7 節
	建構與 Hermite 插值	第 8 節
近似	擬合函數與折線	第 9 節
	描繪等值線	第 10 節
輸出	取樣、JSON、SVG、TikZ、Matplotlib	第 11 節
	命令列	第 12 節

初次使用時，先讀第 3 節與第 5 節。本手冊的圖本身就是 bezierkit 的輸出：每條曲線都由第 11 節的 TikZ 匯出器寫出，再以 LaTeX 編譯。

繪製本手冊各圖的腳本依主題分類，可一次下載：bezierkit-examples.zip。

第 2 節　安裝

2.1　系統需求

bezierkit 需要 Python 3.10 以上版本與 NumPy。另有兩組選用依賴：

`cli` `bezierkit` 指令（詳見第 12 節）；安裝 Typer 與 Rich。

`matplotlib` 與 Matplotlib `Path` 物件互相轉換（詳見第 11.6 節）。

2.2　安裝套件

```
uv add bezierkit                  # the library
uv add "bezierkit[cli]"           # with the command-line interface
uv add "bezierkit[matplotlib]"    # with the Matplotlib adapter
uv add "bezierkit==1.0.0"         # the version this manual describes
```

使用 pip 時，執行 `python -m pip install bezierkit`，選用依賴的寫法相同。1.0.0 是第一個正式版：`bezierkit` 及其文件所列子套件匯出的名稱依語意化版本管理，JSON 路徑格式維持第 1 版。

2.3　開發環境設定

```
git clone https://github.com/EconViz/bezierkit.git
cd bezierkit
uv sync --all-extras --dev
uv run pytest
uv run ruff check src tests benchmarks
uv run lint-imports
```

持續整合流程在 Python 3.10、3.11、3.12 與 3.13 上執行相同指令。`lint-imports` 檢查套件分層：數學核心不匯入任何繪圖端，只有轉接器匯入 Matplotlib。

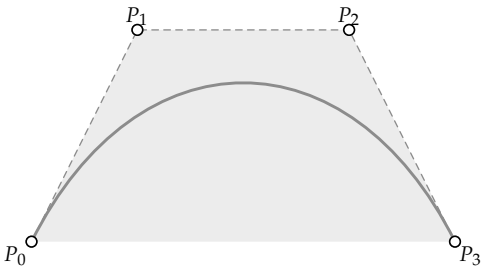


圖 1
三次曲線及其控制凸包。

第 3 節　快速開始

3.1　一條三次曲線

本手冊通篇使用的曲線，是控制點為 $(0,0)$、$(1,2)$、$(3,2)$、$(4,0)$ 的三次曲線（參見圖 1）。

例 1

```python
from bezierkit import BezierCurve, PiecewiseBezier, Point
from bezierkit.bezier import to_cubic
from bezierkit.export.tikz import to_tikz

curve = BezierCurve.cubic(Point(0, 0), Point(1, 2), Point(3, 2), Point(4, 0))
print(curve.at(0.5))                 # Point(coords=(2.0, 1.5))
print(curve.derivative().at(0.5))    # Point(coords=(4.5, 0.0))
left, right = curve.split(0.4)       # two cubics that together trace curve

path = PiecewiseBezier([to_cubic(curve)])
print(to_tikz(path, precision=2, options="thick"))
# \draw[thick] (0.00,0.00) .. controls (1.00,2.00) and (3.00,2.00) .. (4.00,0.00);
```

曲線由 P_0 出發、在 P_3 結束，離開 P_0 時朝向 P_1，抵達 P_3 時來自 P_2 的方向，且始終位於控制點的凸包內（詳見推論 7）。最後一行是這條曲線的 TikZ 路徑，可直接貼進 LaTeX 文件。

3.2　主要元件

- 數值物件：`Point`、`Vector` 與 `PointSet` 不可變，並檢查維度（詳見第 4 節）。
- 曲線：`BezierCurve` 可為任意次數與維度；`CubicBezierSegment` 是繪圖端使用的三次曲線；`PiecewiseBezier` 將三次曲線連接成路徑（詳見第 5 節、第 7 節）。
- 建構：把端點條件、斜率、導數或 Hermite 資料轉為控制點（詳見第 8 節）。
- 近似：把平滑函數、取樣點與等值線 $F(x,y)=c$ 轉為誤差已知的三次路徑（詳見第 9 節、第 10 節）。
- 輸出：取樣、JSON、SVG 路徑資料、TikZ，以及 Matplotlib 路徑（詳見第 11 節）。

第 4 節　幾何數值物件

所有數值物件都不可變，運算會回傳新物件。座標必須為有限值；建立物件時遇到 NaN 或無窮大會拋出 `ValueError`。

4.1　點與向量

`Point`　`Point(⟨浮點數⟩, ...)`

`Vector`　`Vector(⟨浮點數⟩, ...)`

$\mathbb{R}^d$ 中的點或向量，由 $d \geq 1$ 個座標給定。點與向量依仿射幾何的規則運算（參見表 2）。

表 2　點與向量的運算

運算式	結果
`point - point`	`Vector`
`point + vector`、`point - vector`	`Point`
`vector + vector`、`vector * scalar`	`Vector`；也可寫成 `scalar * vector`

`coords`　座標 tuple。

`dimension`　維度 d。

`x`　第一個座標（存在時）。

`y`　第二個座標（存在時）。

`z`　第三個座標（存在時）。

`dot()`　兩個向量的內積。

`norm()`　向量的歐氏長度。

`normalized()`　與向量同向的單位向量；零向量無法正規化。

維度不同時拋出 `DimensionMismatch`。`point + point` 不會被拒絕：它把座標相加並回傳 `Point`，只有在權重總和為一的組合中才有意義，例如 de Casteljau 演算法中的平均。

```
from bezierkit import Point, Vector

p = Point(1, 2)
q = p + Vector(3, -1) * 2      # Point(coords=(7.0, 0.0))
v = q - p                      # Vector(coords=(6.0, -2.0))
print(v.norm())                # 6.324555320336759
```

4.2　點集合

`PointSet`　`PointSet(⟨控制點⟩)`

一批不可變的點，背後是唯讀的 (count, d) NumPy 陣列，批次求值與取樣都回傳此物件。提供 `count`、`dimension`、`array`（唯讀副本）、各欄 `x`、`y`、`z`，可逐一取出 `Point`，也可用索引存取。

4.3　參數

`Interval`　`Interval(start, end)` 是閉區間，提供 `contains()`、`clamp()` 與 `linspace()`。

`ParameterValues`　依定義域檢查單一參數或一維參數陣列。每條曲線都使用它，因此對定義域為 $[0, 1]$ 的曲線呼叫 `at(1.2)` 會拋出 `ParameterOutOfDomain`。

4.4　例外

表 3　例外類別

例外	拋出時機
`BezierKitError`	下列例外的基礎類別

例外	拋出時機
`DimensionMismatch`	組合不同維度的點、向量或線段
`DegreeError`	曲線次數不適用於某項運算，或沒有控制點
`ParameterOutOfDomain`	參數超出曲線定義域 $[0,1]$
`ToleranceNotMet`	自適應擬合在限制內無法達到容許誤差（詳見第 9 節）

不屬於幾何的無效參數（例如負的容許誤差）拋出 `ValueError`。

第 5 節　Bézier 曲線

5.1　Bernstein 基底

定義 1 (Bernstein 多項式) $n \geq 0$ 次 Bernstein 多項式為

$$b_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}, \quad i = 0, \dots, n.$$

其他整數 i 令 $b_{i,n} = 0$。

這組多項式出自 Bernstein (1912)；Farouki (2012)回顧其歷史與性質，三次的情形如圖 2。以下證明用到兩個二項式係數恆等式。

引理 2 (二項式恆等式) 令 $n \geq 1$，i、j 為整數，並約定 $k < 0$ 或 $k > m$ 時 $\binom{m}{k} = 0$。則

$$\binom{n-1}{j} + \binom{n-1}{j-1} = \binom{n}{j}$$

且

$$i\binom{n}{i} = n\binom{n-1}{i-1}.$$

命題 3 (Bernstein 基底) 對 $n \geq 0$，多項式 $b_{0,n}, \dots, b_{n,n}$ 是次數不超過 n 的實係數多項式向量空間的一組基底。

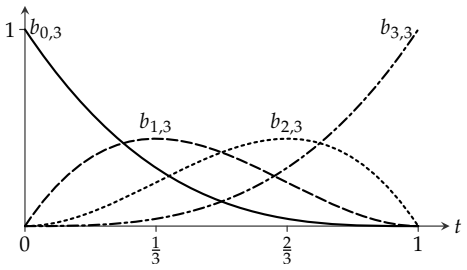


圖 2　三次 Bernstein 多項式。

這是 Floater (2025) 的定理 1.1；附錄 A 從頭證明。

命題 4 (單位分割) *對每個 $t \in [0,1]$，所有 $b_{i,n}(t) \geq 0$，且*

$$\sum_{i=0}^{n} b_{i,n}(t) = 1.$$

定義 5 (Bézier 曲線) 設 $P_0, \ldots, P_n \in \mathbb{R}^d$。以 $P_0, \ldots, P_n$ 為控制點的 n 次 Bézier 曲線為

$$B(t) = \sum_{i=0}^{n} b_{i,n}(t) P_i, \quad t \in [0,1],$$

其控制多邊形為折線 $P_0 P_1 \ldots P_n$。

此定義依循 Bézier (1966)，形式如 Farin (2002) 與 Prautzsch et al. (2002)。

命題 6 (端點) *$b_{i,n}(0)$ 在 $i = 0$ 時為 1，其餘為 0；$b_{i,n}(1)$ 在 $i = n$ 時為 1，其餘為 0。因此 $B(0) = P_0$、$B(1) = P_n$。*

推論 7 (凸包) *每個 $B(t)$（$t \in [0,1]$）都位於控制點 $P_0, \ldots, P_n$ 的凸包內。*

命題 8 (仿射不變性) *對每個仿射映射 $A(x) = Mx + v$ 與每個 $t \in [0,1]$，*

$$A(B(t)) = \sum_{i=0}^{n} b_{i,n}(t) A(P_i).$$

變換曲線等同於變換其控制點。

推論 7 讓曲線不超出控制多邊形所張的區域；命題 8 則是匯出器與 Matplotlib 轉接器只需移動控制點，就能平移、縮放或旋轉曲線的原因。

BernsteinBasis `BernsteinBasis(⟨整數⟩)(⟨浮點數⟩)`

`BernsteinBasis(⟨整數⟩).matrix(⟨數值⟩)`

某一次數的基底，位於 `bezierkit.bezier.basis`。在 t 呼叫時，以陣列回傳 $i = 0, \ldots, n$ 的 $b_{i,n}(t)$；`matrix()` 對每個參數值回傳一列。

5.2　曲線

BezierCurve `BezierCurve(⟨控制點⟩, *, evaluator=None)`

`BezierCurve.linear(p0, p1)`

`BezierCurve.quadratic(p0, p1, p2)`

`BezierCurve.cubic(p0, p1, p2, p3)`

不可變的 Bézier 曲線，次數 $n \geq 0$、維度 $d \geq 1$ 均不限，參數範圍為 $[0,1]$。`points` 可為 `Point` 序列、`PointSet`、$(n+1)\times d$ 陣列或 `ControlPolygon`；所有控制點必須同一維度。

`degree`	次數 n。
`dimension`	維度 d。
`control_points`	以 `PointSet` 表示的控制點。
`at()`	以 `Point` 回傳 $B(t)$；曲線物件也可直接呼叫，`curve(t)`。
`at_many()`	以 `PointSet` 回傳多個參數下的值。
`evaluator`	求值策略（見下文）。

曲線的運算 `derivative()`、`split()`、`segment()` 與 `reversed()` 詳見第 6 節。

5.3　求值

de Casteljau 演算法以反覆的線性插值計算 $B(t)$。

定義 9 (de Casteljau 點) 對參數 t，令 $P_i^{(0)} = P_i$，並對 $r = 1, \dots, n$ 令

$$P_i^{(r)} = (1-t)P_i^{(r-1)} + tP_{i+1}^{(r-1)}, \quad i = 0, \dots, n-r.$$

每一輪對相鄰點取平均，多邊形少一個點；n 輪後只剩一點（參見圖 3）。此演算法源自 Paul de Casteljau 約 1959 年在 Citroën 的工作 (Müller, 2024)。

定理 10 (de Casteljau) 對 $0 \leq r \leq n$ 與 $0 \leq i \leq n-r$，

$$P_i^{(r)} = \sum_{j=0}^{r} b_{j,r}(t)\, P_{i+j}.$$

特別地，$P_0^{(n)} = B(t)$。

此敘述見 Floater (2025) 的定理 1.6 與 1.7。

每個中間點都是控制點的凸組合，演算法不會產生互相抵消的大數，因此在高次數時數值穩定。每個參數的計算量為 $O(n^2)$。

`DeCasteljauEvaluator`　預設的求值策略，位於 `bezierkit.bezier.evaluation`：以 NumPy 一次對整批參數執行上述演算法。

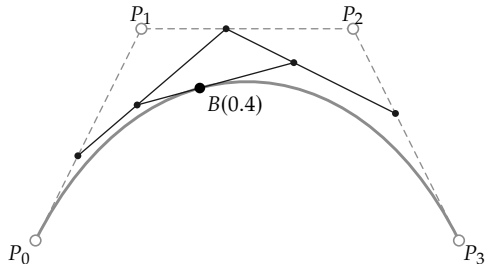


圖 3　de Casteljau 演算法 (t = 0.4)。

`BernsteinEvaluator` 以 Bernstein 矩陣乘上控制點。次數低、批次大時較快，但高次數時會加總可能互相抵消的項。依定理 10，兩種策略計算的是同一個多項式。

```
from bezierkit import BezierCurve
from bezierkit.bezier.evaluation import BernsteinEvaluator

fast = BezierCurve(curve.control_points,
evaluator=BernsteinEvaluator())
print(fast.at(0.5))   # Point(coords=(2.0, 1.5)), as with the default
```

儲存庫的 `benchmarks/` 目錄比較兩者在 400、10,000 與 100,000 個參數值下的效能。

5.4　升階

n 次曲線也是 $n+1$ 次曲線；新的控制點是舊控制點中相鄰兩點的凸組合。

引理 11 (升階恆等式) *對 $n \geq 0$、$0 \leq i \leq n$ 與每個 t，*

$$b_{i,n}(t) = \frac{n+1-i}{n+1} b_{i,n+1}(t) + \frac{i+1}{n+1} b_{i+1,n+1}(t).$$

命題 12 (升階) *設 B 的控制點為 $P_0, \dots, P_n$。對 $k = 0, \dots, n+1$ 令*

$$Q_k = \frac{k}{n+1} P_{k-1} + \left(1 - \frac{k}{n+1}\right) P_k,$$

其中 P_{-1} 與 P_{n+1} 可任意選取，因為它們的係數為零。則

$$B(t) = \sum_{k=0}^{n+1} b_{k,n+1}(t)\, Q_k \quad \text{對所有 } t.$$

曲線及其參數化都不變。

此公式是標準結果 (Farin, 2002; Prautzsch et al., 2002)；附錄 A 的證明不依賴它們。命題 22 把它用在直線與二次曲線，也就是本套件需要的情形。

第 6 節　導數、反轉與分割

每項運算只作用於控制點，並回傳沿用原曲線求值策略的新曲線。

6.1　導數

Bézier 曲線的導數 $B'(t)$ 是其座標函數（t 的多項式）的導數，為 $\mathbb{R}^d$ 中的向量。

引理 13 (Bernstein 多項式的導數) *對 $n \geq 1$ 與每個整數 i，*

$$b'_{i,n}(t) = n\big[b_{i-1,n-1}(t) - b_{i,n-1}(t)\big].$$

此為 Floater (2025) 的引理 1.4。

定理 14 (速端曲線) *n 次 Bézier 曲線（$n \geq 1$）的導數是 $n-1$ 次 Bézier 曲線*

$$B'(t) = \sum_{i=0}^{n-1} b_{i,n-1}(t) \cdot n(P_{i+1} - P_i).$$

控制點為 $n(P_{i+1} - P_i)$ 的曲線稱為 B 的速端曲線 (Floater (2025) 的定理 1.8；另見 Farin (2002))。

推論 15 (端點切線) $B'(0) = n(P_1 - P_0)$，$B'(1) = n(P_n - P_{n-1})$。

這說明了曲線為何朝 P_1 的方向離開 P_0，並從 P_{n-1} 的方向抵達 P_n。

`BezierCurve.derivative` `derivative(order=1)`
以 `BezierCurve` 回傳 `order` 階導數，即套用定理 14 `order` 次。常數（0 次）的導數是 0 次的零曲線；`order=0` 回傳曲線本身。

```
d = curve.derivative()
print(list(d.control_points))
# [Point(coords=(3.0, 6.0)), Point(coords=(6.0, 0.0)),
Point(coords=(3.0, -6.0))]
print(d.at(0.5))   # Point(coords=(4.5, 0.0)): the tangent at the top
is horizontal
```

6.2　反轉

引理 16 (對稱性) *對所有 i 與 t，$b_{i,n}(1-t) = b_{n-i,n}(t)$。*

命題 17 (反轉) *控制點為 $P_n, \ldots, P_0$ 的曲線就是 $t \mapsto B(1-t)$。*

`BezierCurve.reversed` 依命題 17，回傳反向走訪的曲線。

6.3　分割

在 $t = c$ 執行 de Casteljau 演算法不只得到曲線值：每一輪的第一個點構成 c 之前那段曲線的控制多邊形，最後一個點構成 c 之後那段的控制多邊形（參見圖 4）。

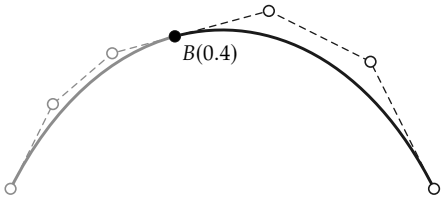


圖 4
在 $t = 0.4$ 分割三次曲線。

定理 18 (分割) 令 $c \in [0,1]$，並在 $t = c$ 計算定義 9 的各點。令 $L_j = P_0^{(j)}$、$R_j = P_j^{(n-j)}$，$j = 0, \ldots, n$。則對每個 $s \in [0,1]$

$$B(cs) = \sum_{j=0}^{n} b_{j,n}(s) L_j$$

且

$$B(c + (1-c)s) = \sum_{j=0}^{n} b_{j,n}(s) R_j.$$

這是標準結果，見 Floater (2025)（第 8.4 節，由 blossom 導出）與 Farin (2002)。

`BezierCurve.split` `split(⟨浮點數⟩)`

`BezierCurve.segment` `segment(t0, t1)`

`split(c)` 回傳定理 18 的兩條曲線，次數不變，參數範圍都是 $[0,1]$。`segment(t0, t1)` 回傳曲線在 t_0 與 t_1 之間的部分，重新參數化到 $[0,1]$；$t_0 = t_1$ 時回傳單點 $B(t_0)$，即 0 次曲線。參數超出 $[0,1]$ 或 $t_0 > t_1$ 時拋出例外。

`segment()` 分割兩次：先在 t_1 分割並保留左段，再在 t_0/t_1 分割該段並保留右段。

推論 19 (截取) 對 $0 \le t_0 < t_1 \le 1$，*`segment(t0, t1)` 回傳的曲線為* $s \mapsto B(t_0 + (t_1 - t_0)s)$。

```
left, right = curve.split(0.4)
print(left.at(1.0), right.at(0.0))          # both B(0.4) = (1.552,
1.44)
print(curve.segment(0.25, 0.75).at(0.5))    # B(0.5) = (2.0, 1.5)
```

第 7 節　三次線段與路徑

繪圖端畫的是三次曲線：TikZ 的 `.. controls ..`、SVG 的 `C` 與 Matplotlib 的 `CURVE4` 都接收四個控制點。`CubicBezierSegment` 就是以數值物件表示的三次曲線，`Piecewise Bezier` 則把三次曲線連接成路徑。

7.1　三次線段

`CubicBezierSegment` `CubicBezierSegment(p0, p1, p2, p3, *, fit_error=None)`

不可變的三次曲線，四個控制點即欄位 `p0`、`p1`、`p2`、`p3`（亦可用 `control_points` 或逐一取出）。求值、微分、分割、截取與反轉的方式與 `BezierCurve` 相同，但 `split()`、`segment()` 與 `reversed()` 仍回傳三次線段。`segment(t, t)` 回傳四個控制點都是 $B(t)$ 的退化三次曲線。

`fit_error` 擬合線段的實測誤差（詳見第 9 節）；分割、截取與反轉後保留，判斷相等時忽略。

float | *None* · 預設 `None`

`as_curve()` 轉為 3 次 `BezierCurve`，不改變任何控制點。

`from_curve(curve)` 由 3 次 `BezierCurve` 轉換，不改變任何控制點。

`from_line()` 直線的精確三次表示（詳見命題 22）。

`from_quadratic()` 二次曲線的精確三次表示（詳見命題 22）。

7.2　緊密邊界框

定義 20 (軸對齊邊界框) 非空有界集合 $K \subset \mathbb{R}^d$ 的軸對齊邊界框，是包含 K 的最小方框

$$[\alpha_1, \beta_1] \times \ldots \times [\alpha_d, \beta_d],$$

其中 $\alpha_k = \inf\{x_k : x \in K\}$、$\beta_k = \sup\{x_k : x \in K\}$。

控制點凸包包含曲線（詳見推論 7），但可能比曲線大得多。`bounding_box` 回傳曲線本身的定義 20 所定義的方框（參見圖 5），其候選點來自各座標導數的根。

命題 21 (三次曲線的邊界框) 設 B 是控制點為 $P_0, \ldots, P_3 \in \mathbb{R}^d$ 的三次曲線，令

$$a = -P_0 + 3P_1 - 3P_2 + P_3, \quad b = 3P_0 - 6P_1 + 3P_2, \quad c = 3(P_1 - P_0).$$

則 $B(t) = at^3 + bt^2 + ct + P_0$。對每個座標 k，令 T_k 由 0、1 與下式在 $(0,1)$ 內的根組成

$$B_k'(t) = 3a_k t^2 + 2b_k t + c_k$$

（此多項式不恆為零時）；若它恆為零，則 $T_k = \{0, 1\}$。於是 $B([0,1])$ 的邊界框座標為

$$\alpha_k = \min_{t \in T_k} B_k(t), \quad \beta_k = \max_{t \in T_k} B_k(t).$$

`CubicBezierSegment.bounding_box` 回傳 `(low, high)` 兩點，分別為曲線在各軸的最小值與最大值，由命題 21 的候選參數求值而得。

7.3　升階

升階（詳見命題 12）把 n 次曲線寫成參數化相同的 $n+1$ 次曲線。本套件用它把直線與二次曲線寫成三次曲線，使每個繪圖端都收到四個控制點。對命題 12 套用一次或兩次，即得下列控制點。

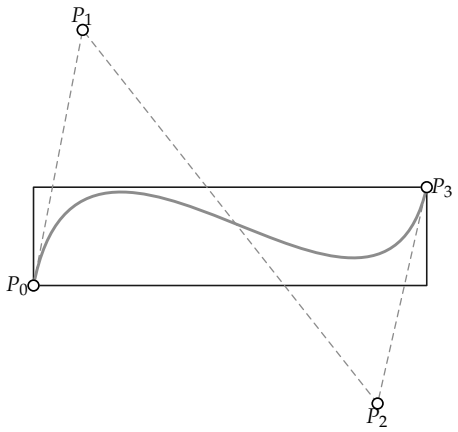


圖 5
三次曲線的緊密邊界框。

命題 22 (直線與二次曲線的三次表示) 由 P_0 到 P_3、參數化為 $(1-t)P_0 + tP_3$ 的直線，是控制點為

$$P_0,\quad P_0 + \frac{1}{3}(P_3 - P_0),\quad P_0 + \frac{2}{3}(P_3 - P_0),\quad P_3$$

的三次曲線。控制點為 P_0, C, P_3 的二次曲線，是控制點為

$$P_0,\quad P_0 + \frac{2}{3}(C - P_0),\quad P_3 + \frac{2}{3}(C - P_3),\quad P_3$$

的三次曲線。兩種情形下曲線與其參數化都不變。

`to_cubic` `to_cubic(⟨曲線⟩)`
位於 `bezierkit.bezier`。依命題 22 將一次、二次或三次 `BezierCurve` 轉為 `CubicBezierSegment`（傳入線段時原樣回傳）；更高次數拋出 `DegreeError`。

`line_to_cubic` `line_to_cubic(p0, p3)`
由 `p0` 到 `p3` 的直線的三次表示。

`quadratic_to_cubic` `quadratic_to_cubic(p0, control, p3)`
控制點如給定的二次曲線的三次表示。

```
from bezierkit.bezier import to_cubic

q = BezierCurve.quadratic(Point(0, 0), Point(3, 6), Point(9, 0))
print(to_cubic(q).control_points)
# (Point(coords=(0.0, 0.0)), Point(coords=(2.0, 4.0)),
#  Point(coords=(5.0, 4.0)), Point(coords=(9.0, 0.0)))
```

7.4　分段路徑

`PiecewiseBezier` `PiecewiseBezier(⟨segments⟩, *, closed=False, continuity_tolerance=1e-9)`

`PiecewiseBezier.compound(⟨paths⟩)`

由三次線段組成的路徑。相鄰線段必須相接：每段的 `p3` 與下一段的 `p0` 距離不得超過 `continuity_tolerance`，封閉路徑最後的 `p3` 與第一個 `p0` 亦然，否則拋出 `ValueError`。`closed` 是給匯出器的資訊（SVG 的 `Z`、TikZ 的 `cycle`），不會額外加入封閉線段。

本套件以均勻方式將路徑參數化。這是本套件的約定，並非通用概念：由 m 段 $S_0, \dots, S_{m-1}$ 組成的路徑是映射

$$P(t) = S_k(mt - k), \quad k = \min(\lfloor mt \rfloor, m-1), \quad t \in [0,1].$$

不論長短，每段都占 $[0,1]$ 的 $1/m$。

`segments`	依序排列的線段。
`control_points`	其控制點。
`subpaths`	各 `BezierSubpath`。
`at(t)`	依均勻參數化求值。
`at_many(values)`	在多個參數上求值。
`split(t)`	在 `t` 分割路徑；參數落在某段內時會分割該段（詳見定理 18）。
`segment(t0, t1)`	保留兩個參數之間的部分。
`reversed()`	反轉線段順序與每一段。
`closed`	封閉旗標。
`is_compound`	是否含多條子路徑。
`dimension`	維度 d。

```
from bezierkit import CubicBezierSegment, PiecewiseBezier

path = PiecewiseBezier([
    CubicBezierSegment.from_line(Point(0, 0), Point(2, 0)),
    CubicBezierSegment.from_line(Point(2, 0), Point(2, 4)),
])
print(path.at(0.75))                      # Point(coords=(2.0, 2.0))
print(path.segment(0.25, 0.75).at(1.0))   # Point(coords=(2.0, 2.0))
```

`compound()` 把多條路徑合為一條，各子路徑保持分開，例如有洞的形狀。複合路徑沒有單一的參數化：求值、分割或截取會拋出 `ValueError`；反轉與匯出則可正常使用。分割或截取封閉路徑也會拋出例外（整條路徑或單點除外），因為封閉路徑沒有可保留的起點與終點。

7.5　連續性

兩條曲線在接點相接。設 $h_S, h_T > 0$，$S : [u_0 - h_S, u_0] \to \mathbb{R}^d$ 與 $T : [u_0, u_0 + h_T] \to \mathbb{R}^d$ 具有所需階數的連續導數（區間端點取單邊導數），C 為在前一區間等於 S、在後一區間等於 T 的曲線。

定義 23 (參數連續性) 對 $k \geq 0$，若 $S^{(j)}(u_0) = T^{(j)}(u_0)$ 對 $j = 0, \dots, k$ 成立，稱曲線 C 在 u_0 為 C^k。

定義 24 (幾何連續性 G^1) 若 $S(u_0) = T(u_0)$，切向量 $S'(u_0)$ 與 $T'(u_0)$ 皆非零，且存在 $\lambda > 0$ 使 $T'(u_0) = \lambda S'(u_0)$，稱曲線 C 在 u_0 為 G^1；也就是兩側的單位切向量相同。

參數連續性與幾何連續性是連接曲線的標準概念 (Farin, 2002; Prautzsch et al., 2002)；任意階 k 的幾何連續性見 Barsky & DeRose (1989)。本手冊只用到 C^0、C^1 與 G^1。切向量非零的 C^1 接點必為 G^1。

命題 25 (三次曲線接點的連續性) 設 S、T 是控制點分別為 $P_0, \dots, P_3$ 與 $Q_0, \dots, Q_3$ 的三次曲線，並分別在 $(u - u_0 + h_S)/h_S$ 與 $(u - u_0)/h_T$ 求值。

1. C 在 u_0 為 C^0，若且唯若 $P_3 = Q_0$。
2. 若 $P_3 = Q_0$，則 C 在 u_0 為 C^1，若且唯若

$$\frac{P_3 - P_2}{h_S} = \frac{Q_1 - Q_0}{h_T}.$$

3. 若 $P_3 = Q_0$、$P_3 \neq P_2$ 且 $Q_1 \neq Q_0$，則 C 在 u_0 為 G^1，若且唯若存在 $\mu > 0$ 使 $Q_1 - Q_0 = \mu(P_3 - P_2)$。
4. 在 `PiecewiseBezier` 的均勻參數化下 $h_S = h_T$，因此第 2 項的條件即為 $P_3 - P_2 = Q_1 - Q_0$。

`PiecewiseBezier` 只要求 C^0，也就是端點相接。接點是否也要平滑由呼叫端決定：無異曲線的折角或折線的轉角本來就該保留。

第 8 節　建構與 Hermite 插值

三次曲線由兩個端點及其導數決定。本章的建構方法把這類端點條件轉為控制點，經濟學中只知道幾個數值與斜率的曲線，就是這樣變成 Bézier 曲線。

8.1　端點建構

每種建構都是帶有 `build()` 方法的不可變 dataclass，位於 `bezierkit.construction`。

`EndpointDerivatives` `EndpointDerivatives(start, end, start_derivative, end_derivative).build()`
滿足 $B(0) =$ `start`、$B(1) =$ `end`、$B'(0) =$ `start_derivative`、$B'(1) =$ `end_derivative` 的三次曲線，維度不限。

命題 26 (端點條件) 設 $P_0, P_3, D_0, D_1 \in \mathbb{R}^d$。控制點為

$$P_0, \quad P_0 + D_0/3, \quad P_3 - D_1/3, \quad P_3$$

的三次 *Bézier* 曲線滿足 $B(0) = P_0$、$B(1) = P_3$、$B'(0) = D_0$、$B'(1) = D_1$，且它是唯一滿足這些條件、次數不超過 3 的多項式曲線。

`TangentDirections` `TangentDirections(start, end, start_direction, end_direction, start_handle=1.0, end_handle=1.0).build()`

以方向給定端點切線，控制柄長度為 $|P_1 - P_0| =$ `start_handle`、$|P_3 - P_2| =$ `end_handle`。方向會先正規化，因此只有方向有影響；控制柄長度必須為有限的非負值。依命題 26，$B'(0)$ 等於 3 倍 `start_handle` 乘以起點的單位方向。

`PlanarSlopes` `PlanarSlopes(start, end, start_slope, end_slope, start_handle=1.0, end_handle=1.0).build()`

兩端斜率 $\mathrm{d}y/\mathrm{d}x$ 為給定值的平面三次曲線，以方向 $(1, m)$ 的 `TangentDirections` 建構。斜率必須為有限值；垂直切線請改用 `TangentDirections`。

```
from bezierkit.construction import PlanarSlopes

demand = PlanarSlopes(
    start=Point(0, 5), end=Point(5, 0), start_slope=-2, end_slope=-0.3,
).build()
print(demand.control_points[1])   # (0.447, 4.106): one unit from (0, 5) along slope -2
```

8.2　Hermite 插值

三次 Hermite 插值在區間兩端同時吻合函數值與導數。

定義 27 (三次 Hermite 插值多項式) 設 f 在 $[x_0, x_1]$ 上可微，$x_0 \neq x_1$。f 的三次 Hermite 插值多項式是次數不超過 3、滿足

$$H(x_i) = f(x_i), \quad H'(x_i) = f'(x_i), \quad i = 0, 1$$

的多項式 H。對曲線則逐一插值每個座標。

這是 Burden & Faires (2011) 第 3.4 節中 Hermite 多項式取兩個節點的情形，該書的定義 3.8。這樣的多項式存在且唯一：

命題 28 (存在性與唯一性) *定義 27 的三次 Hermite 插值多項式存在且唯一。令 $h = x_1 - x_0$、$f_i = f(x_i)$、$m_i = f'(x_i)$，則*

$$H(x) = \sum_{i=0}^{3} b_{i,3}((x - x_0)/h) c_i,$$

其中係數為

$$c_0 = f_0, \quad c_1 = f_0 + h m_0/3, \quad c_2 = f_1 - h m_1/3, \quad c_3 = f_1.$$

任意節點數的同一結論，見 Stoer & Bulirsch (2002) 的定理 (2.1.5.2)。在寬度為 $h = t_1 - t_0$ 的參數區間 $[t_0, t_1]$ 上，Bézier 線段在 $s = (t - t_0)/h$ 處求值，導數因此乘上 h。

`parametric_hermite` `parametric_hermite(p0, p3, derivative0, derivative1, *, t0=0.0, t1=1.0)`

控制點為 P_0、$P_0 + hD_0/3$、$P_3 - hD_1/3$、P_3 的三次線段，位於 `bezierkit.interpolation`。$h = 0$ 時拋出 `ValueError`；h 為負表示反向的區間。

命題 29 (Hermite 插值) 對 *parametric_hermite* 回傳的線段，令 $H(t) = B((t-t_0)/h)$。則 $H(t_0) = P_0$、$H(t_1) = P_3$、$H'(t_0) = D_0$、$H'(t_1) = D_1$。

graph_hermite

`graph_hermite(*, x0, x1, y0, y1, m0, m1)`

由函數值 $y_0 = f(x_0)$、$y_1 = f(x_1)$ 與斜率 $m_0 = f'(x_0)$、$m_1 = f'(x_1)$ 得到圖形 $y = f(x)$ 的插值曲線，即取 $P = (x, f(x))$、$D = (1, f'(x))$、$t = x$ 的參數形式。控制點為

$$(x_0, y_0),\quad \left(x_0 + \frac{h}{3}, y_0 + hm_0/3\right),\quad \left(x_1 - \frac{h}{3}, y_1 - hm_1/3\right),\quad (x_1, y_1),$$

其中 $h = x_1 - x_0$（參見圖 6）。

命題 30 (圖形形式) *graph_hermite* 的線段是 f 在 $[x_0, x_1]$ 上的三次 *Hermite* 插值多項式 H 的圖形：即 $s \mapsto (x_0 + hs, H(x_0 + hs))$，$s \in [0,1]$。

```
from bezierkit.interpolation import graph_hermite

# f(x) = 4 / x^2 on [1, 1.5]
segment = graph_hermite(x0=1, x1=1.5, y0=4, y1=16 / 9, m0=-8, m1=-64 /
27)
```

8.3　插值誤差

定理 31 (Hermite 誤差界限) 設 f 在 $[x_0, x_1]$ 上四階連續可微，$x_0 < x_1$，H 為其三次 *Hermite* 插值多項式（詳見定義 27），令 $h = x_1 - x_0$、$M = \max|f^{(4)}|$。對每個 $x \in [x_0, x_1]$，存在 $\xi \in [x_0, x_1]$ 使

$$f(x) - H(x) = \frac{f^{(4)}(\xi)}{24}(x - x_0)^2 (x - x_1)^2,$$

因此

$$\max_{x \in [x_0, x_1]} |f(x) - H(x)| \le \frac{Mh^4}{384}.$$

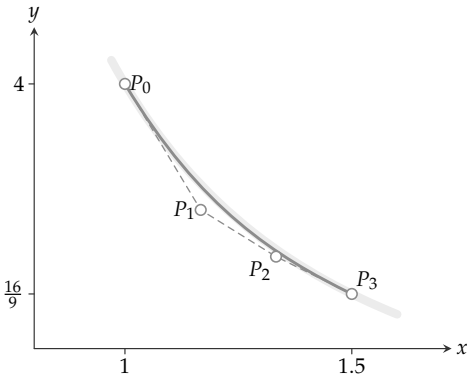


圖 6 $4/x^2$ 在 $[1, 1.5]$ 的 Hermite 插值。

誤差公式見 Burden & Faires (2011) 的定理 3.9（其證明留作該書習題 3.4.11）與 Stoer & Bulirsch (2002) 的定理 (2.1.5.9)。此界限是銳利的：對 $[0,1]$ 上的 $f(x)=x^4$，誤差為 $x^2(x-1)^2$，於 $x=1/2$ 的最大值 $1/16$ 等於 $M=24$ 時的 $Mh^4/384$。

區間減半時界限變為 $1/16$，自適應擬合正是利用這一點（詳見第 9 節）。以 $f(x)=4/x^2$ 在 $[1,1.5]$ 為例，$f^{(4)}(x)=480/x^6$，故 $M=480$，界限為 $480\cdot 0.5^4/384\approx 0.078$。

`hermite_error_bound` `hermite_error_bound(max_fourth_derivative, t0, t1)`

定理 31 的界限 $M\,|t_1-t_0|^4\,/384$，位於 `bezierkit.fitting`。M 必須為有限的非負值。

此界限適用於純量函數。對曲線 $t\mapsto(x(t),y(t))$，分別套用到每個座標。在 $\mathbb{R}^d$ 中，歐氏誤差至多為最大座標界限的 $\sqrt{d}$ 倍，因為對 $v\in\mathbb{R}^d$ 有 $\|v\|_2\le\sqrt{d}\,\|v\|_\infty$（見 Golub & Van Loan (2013) 第 2.2 節）。

第 9 節　擬合

擬合把平滑函數或一串取樣點轉為 `PiecewiseBezier`，並逐段量測、回報誤差，單位與曲線座標相同。相關函式位於 `bezierkit.fitting`。

9.1　自適應 Hermite 擬合

參數曲線 $C:[t_0,t_1]\to\mathbb{R}^d$ 在區間 $[a,b]$ 上的擬合，是逐一座標取定義 27 的三次 Hermite 插值多項式（詳見命題 29）。套件在 q 個等距的量測參數 $\tau_j=a+j(b-a)/(q-1)$（$j=0,\dots,q-1$，q 為 `error_samples`）上量測誤差：

$$e=\max_{0\le j\le q-1}\bigl\|C(\tau_j)-H(\tau_j)\bigr\|_2 .$$

此實測誤差是本套件的約定，絕不大於線段真實的最大誤差。

`fit_parametric` `fit_parametric(⟨function⟩, ⟨derivative⟩, *, t0, t1, tolerance, max_depth=20, max_segments=4096, error_samples=9)`

在 $[t_0,t_1]$ 上擬合參數曲線 $C(t)$；`function` 回傳 `Point`，`derivative` 回傳 `Vector`。演算法為遞迴二分：

1. 以目前區間兩端的 C 與 C' 建立 Hermite 線段（詳見命題 29）。
2. 量測誤差 e。
3. e 不超過 `tolerance` 時保留此線段，並將 e 記為其 `fit_error`；否則將區間對半，兩半各自擬合。

因此每個保留的線段在量測參數上都符合容許誤差。若區間到了 `max_depth` 仍不合格，或保留它會超過 `max_segments`，就拋出 `ToleranceNotMet`，而不回傳品質較差的路徑。

`tolerance`
float
容許誤差，單位同座標；必須為正。

`max_depth`
int · 預設 20
起始區間最多可對半的次數。

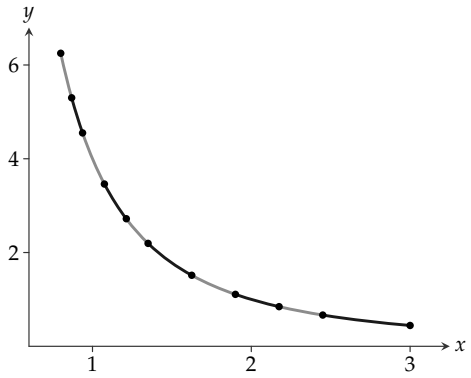


圖 7
$4/x^2$ 的自適應擬合。

`max_segments`
int · 預設 4096
結果最多的線段數。

`error_samples`
int · 預設 9
每段的量測參數個數，至少 3。

`fit_graph`
`fit_graph(⟨function⟩, ⟨derivative⟩, *, x0, x1, tolerance, ...)`
擬合圖形 $y = f(x)$，$x \in [x_0, x_1]$：即以 $C(x) = (x, f(x))$、$C'(x) = (1, f'(x))$ 呼叫 `fit_parametric`，選項相同，誤差為 (x, y) 平面上的歐氏距離。

```
from bezierkit.fitting import fit_graph

path = fit_graph(lambda x: 4 / x**2, lambda x: -8 / x**3,
                 x0=0.8, x1=3, tolerance=1e-3)
print(len(path.segments))                    # 10
print(max(s.fit_error for s in path))        # 0.000676
```

曲線彎曲劇烈處線段較短，接近直線處線段較長（參見圖 7，相鄰線段以不同顏色區分）。Hermite 誤差界限可預估二分需要的深度：

推論 32 (自適應擬合的深度) 設 C 的每個座標在 $[t_0, t_1]$ 上四階連續可微且 $|C_k^{(4)}| \le M$，令 $L = |t_1 - t_0|$，d 為維度，$\varepsilon > 0$ 為容許誤差。令

$$k^* = \max\left(0, \left\lceil \frac{1}{4} \log_2\left(\sqrt{d} M \frac{L^4}{384\varepsilon} \right) \right\rceil\right),$$

$M = 0$ 時 $k^* = 0$。深度 $k \ge k^*$ 的每個區間都能通過誤差檢查。因此若 *max_depth* $\ge k^*$ 且 *max_segments* $\ge 2^{k^*}$，*fit_parametric* 不會拋出 *ToleranceNotMet*，也不會產生深度大於 k^* 的區間。

實測誤差是線段真實最大誤差的下界，因為只在量測參數上檢查。若曲線可能在量測點之間起伏，請增加 `error_samples`。

9.2　折線簡化

定義 33 (點到線段的距離) 對 $x, a, b \in \mathbb{R}^d$，x 到線段 $[a,b]$ 的距離為

$$\operatorname{dist}(x,[a,b]) = \min_{\lambda\in[0,1]} \|x - a - \lambda(b-a)\|_2.$$

以遞迴分割簡化折線，就是 Ramer–Douglas–Peucker 演算法 (Douglas & Peucker, 1973; Ramer, 1972)。`fit_polyline` 全程使用定義 33 的距離。

`fit_polyline` `fit_polyline(⟨控制點⟩, *, tolerance, closed=False, duplicate_tolerance=1e-12, preserve_corners=True, corner_angle=pi/4)`

把取樣點簡化為由直線弦組成的路徑，每條弦以精確的三次曲線表示（詳見命題 22）。步驟如下：

1. 刪除與前一點距離在 `duplicate_tolerance` 以內的點。封閉折線另外刪除結尾重複的起點，並旋轉為從字典序最小的點開始，使結果與取樣起點無關。
2. 保留兩端點；啟用 `preserve_corners` 時，另保留方向轉折至少 `corner_angle` 弧度的頂點。
3. 在相鄰的保留頂點之間套用遞迴步驟：若離弦最遠的頂點距離不超過 `tolerance`，以弦取代整段；否則保留該頂點，兩側遞迴處理。

每條弦的 `fit_error` 是被它取代的頂點到弦的最大距離（參見圖 8）。

遞迴最多有與頂點數相同的層數，每層對每個頂點至多掃描一次，因此最壞情況下與頂點數成平方關係。Hershberger & Snoeyink (1994) 給出此演算法的 $O(n \log n)$ 實作。

命題 34 (折線容許誤差) *設 $v_0, \dots, v_N$ 為通過步驟 1 的頂點（封閉折線有 $v_N = v_0$），$0 = a_0 < \dots < a_m = N$ 為輸出保留的頂點指標，ε 為 `tolerance`。對每個 j 與每個滿足 $a_j \le i \le a_{j+1}$ 的 i，*

$$\operatorname{dist}\Big(v_i, \big[v_{a_j}, v_{a_{j+1}}\big]\Big) \le \varepsilon.$$

特別地，每個輸出線段的 `fit_error` 都不超過 ε。

推論 35 (與折線的偏差) *沿用命題 34 的記號：*

1. *折線 $v_0 v_1 \dots v_N$ 上的每個點，都與輸出的某條弦 $\big[v_{a_j}, v_{a_{j+1}}\big]$ 相距不超過 ε。*
2. *每個輸入點與某條弦相距不超過 $\varepsilon + \delta$，其中 δ 為 `duplicate_tolerance`；通過步驟 1 的點相距不超過 ε。*

`maximum_polyline_deviation` `maximum_polyline_deviation(⟨控制點⟩, ⟨路徑⟩)`

各點到路徑中最近之弦 P_0P_3 的最大距離：可用來檢查命題 34 與推論 35，也涵蓋步驟 1 刪除的點。

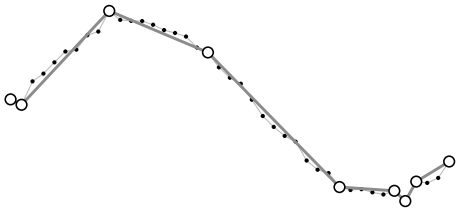


圖 8 以容許誤差 0.08 簡化 41 個雜訊樣本。

如同所有只看得到樣本的方法，`fit_polyline` 限制的是與樣本的偏差，而非與樣本之間未知連續曲線的偏差；需要更貼近時請加密取樣。

第 10 節　等值線

無異曲線、等產量曲線與等高線，都是二元函數的等值集。

10.1　等值集與切線

定義 36 (等值集) 對 $F:\mathbb{R}^2\to\mathbb{R}$ 與水準 $c\in\mathbb{R}$，等值集為

$$L_c=\{(x,y):F(x,y)=c\}.$$

對連續可微的 F，以 F_x、F_y 表示偏導數，$\nabla F=\left(F_x,F_y\right)$ 為梯度。在 $\nabla F\neq 0$ 的點附近，等值集是一條曲線，這由隱函數定理得出；以下引述所需的形式。

定理 37 (隱函數定理) 設 $\Omega\subset\mathbb{R}^2$ 為開集，$F:\Omega\to\mathbb{R}$ 連續可微，$p\in\Omega$，$F(p)=c$ 且 $F_y(p)\neq 0$。則存在開區間 I、J 滿足 $p_x\in I$、$p_y\in J$、$I\times J\subset\Omega$，以及連續可微的 $\varphi:I\to J$、$\varphi(p_x)=p_y$，使得對 $(x,y)\in I\times J$，

$$F(x,y)=c\quad\text{若且唯若}\quad y=\varphi(x).$$

此定理對任意個變數都成立，見 Rudin (1976) 的定理 9.28、Apostol (1974) 第二版的定理 13.7 與 Munkres (1991) 的定理 9.2。此處不予證明。

定理 38 (等值集的切線) 設 F 在開集 $\Omega\subset\mathbb{R}^2$ 上連續可微，$p\in\Omega$，$F(p)=c$ 且 $\nabla F(p)\neq 0$。則存在 p 的開鄰域 N 與開區間 I 上的連續可微曲線 $\gamma:I\to\mathbb{R}^2$，滿足 $\gamma(I)=L_c\cap N$、某個 $u_0\in I$ 使 $\gamma(u_0)=p$、對所有 $u\in I$ 有 $\gamma'(u)\neq 0$，且 $\gamma'(u)$ 平行於 $\gamma(u)$ 處的 $\left(F_y,-F_x\right)$。向量 $\left(F_y,-F_x\right)$ 與 ∇F 正交。

切線 $\left(F_y,-F_x\right)$ 不需除法，因此垂直切線（$F_y=0$）不需要另外處理；圖形形式的斜率 $\mathrm{d}y/\,\mathrm{d}x=-F_x/F_y$ 在該處則為無窮大。

10.2　描繪

`trace_implicit` 不需解出 y 就能把等值集轉為三次路徑，因此可以處理會折返、封閉或分成數段的曲線。

`trace_implicit` `trace_implicit(⟨function⟩, *, levels, viewport, resolution=(101, 101), tolerance=1e-2, gradient=None)`

在矩形 `viewport` $= (x_{\min}, x_{\max}, y_{\min}, y_{\max})$ 內，描繪每個水準 c 的 $F(x,y) = c$，位於 `bezierkit.implicit`。

`function` F，以兩個浮點數呼叫；在每個網格點都必須為有限值，否則拋出 `ValueError`。

`levels` 單一水準，或由有限水準值組成的可迭代物件。

`resolution` 沿 x 與 y 的網格點數，各至少 2。

(int, int) · 預設 `(101, 101)`

`tolerance` 簡化與彎曲線段的容許偏差，單位同座標。

float · 預設 `1e-2`

`gradient` $\nabla F = \left(F_x, F_y\right)$，以 tuple 或 `Vector` 回傳。提供時，直線段會彎成沿曲線切線方向的三次曲線。

callable | None · 預設 `None`

演算法是行進方格法，即 Lorensen & Cline (1987) 行進立方體法的二維類比。網格格子的角點為 $v_{00}, v_{10}, v_{11}, v_{01}$，由左下角起逆時針排列，$F$ 的值分別為 $f_{00}, f_{10}, f_{11}, f_{01}$。角點的值不小於水準 c 時稱為高。

定義 39 (邊上的交點) 設網格的一條邊由 p 到 q，且 $F(p) \geq c > F(q)$ 或 $F(q) \geq c > F(p)$。其交點為 $p + \lambda(q - p)$，其中

$$\lambda = \frac{c - F(p)}{F(q) - F(p)},$$

即 $F - c$ 沿該邊的線性插值之零點。

一條邊有交點，若且唯若它的兩個角點高低不同。

1. 取樣。在 viewport 的規則網格上計算 F。
2. 行進。在每個網格格子中找出四條邊的交點（詳見定義 39）。有兩個交點的格子貢獻一條線段。有四個交點的格子是鞍點：兩個對角角點為高、另兩個為低，由下述規則決定如何連接（參見圖 9）。
3. 縫合。把端點相同的線段串成鏈。回到起點的鏈是封閉的；抵達 viewport 邊界或分支點的鏈是開放的。互不相連的部分保持為不同的路徑。
4. 簡化。以 `tolerance` 對每條鏈呼叫 `fit_polyline`（詳見第 9 節），不保留轉角。
5. 彎曲（提供 `gradient` 時）。把每條直線段換成端點切線沿等值集方向的三次曲線（詳見定理 38），控制柄長為弦長的三分之一。只有當三次曲線與弦的距離不超過 `tolerance` 時才採用；任一端 $\nabla F = 0$ 時維持直線。

鞍點規則使用四個角點值的雙線性插值

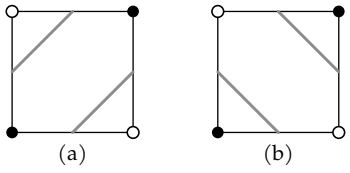


圖 9 鞍點格子：(a) 中心高、(b) 中心低；實心角點的 $F \geq c$。

$$u(s,t) = (1-s)(1-t)f_{00} + s(1-t)f_{10} + stf_{11} + (1-s)tf_{01}, \quad (s,t) \in [0,1]^2.$$

命題 40 (雙線性插值的中心值) 插值 u 在角點取角點值，在格子中心的值等於四個角點值的平均：

$$u\left(\frac{1}{2}, \frac{1}{2}\right) = \frac{f_{00} + f_{10} + f_{11} + f_{01}}{4}.$$

此平均值不小於 c 時，套件將中心視為高。在鞍點格子中，與中心同側的兩個角點在格內相連，另外兩個角點各自被一條線段切開（參見圖 9）。這就是中點決定法（midpoint decider）（Athawale & Johnson, 2019）。Nielson & Hamann (1991) 的漸近決定法（asymptotic decider）則比較 u 在其鞍點的值

$$u^* = \frac{f_{00}f_{11} - f_{10}f_{01}}{f_{00} - f_{10} - f_{01} + f_{11}},$$

兩種規則只在 u^* 與平均值落在 c 兩側時才不同。

```
from bezierkit.implicit import trace_implicit

contours = trace_implicit(
    lambda x, y: x**2 * y,
    levels=[1, 2, 4],
    viewport=(0.5, 4, 0, 6),
    resolution=(121, 121),
    tolerance=0.01,
    gradient=lambda x, y: (2 * x * y, x**2),
)
print(contours.level_values)                    # (1.0, 2.0, 4.0)
print(len(contours.for_level(4)[0].segments))   # 15
```

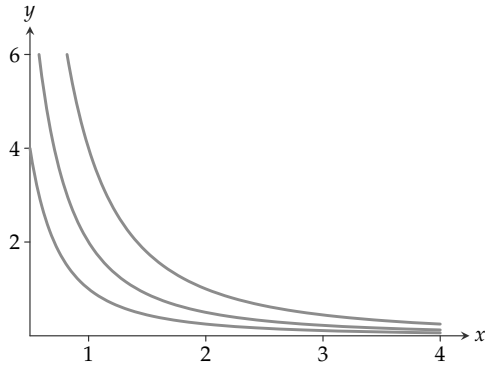


圖 10 $x^2y = c$，$c = 1, 2, 4$ 由內而外。

10.3　結果

`ContourSet` `trace_implicit` 回傳 `ContourSet`，其 `contours` 依給定順序為每個水準保存一個 `LevelContours`。`level_values` 列出各水準，`paths` 匯集所有路徑，`for_level(c)` 回傳某一水準的路徑（未描繪的水準拋出 `KeyError`）。

`LevelContours` 水準值 `level` 與路徑 `paths`，每個連通部分一條 `PiecewiseBezier`。

10.4　限制

描繪出的曲線只在網格交點上精確；交點之間是行進方格法得到的折線，再於 `tolerance` 內簡化與彎曲。需要更貼近時，請提高 `resolution` 並降低 `tolerance`。比網格格子小的特徵可能遺漏，鞍點規則也只會選擇兩種可能連接方式之一。Leontief 無異曲線的折角等尖點會在網格間距內被圓滑化，因為網格看不到精確的轉角。

第 11 節　取樣與匯出

匯出器只寫出幾何資料，不選擇顏色、線寬、主題、座標軸、標籤或畫布，也絕不把三次曲線攤平成折線：TikZ 收到原生的 `.. controls ..` 曲線，SVG 收到原生的 `C` 指令。

11.1　取樣

`UniformSampler` `UniformSampler(⟨整數⟩).sample(⟨曲線⟩)`
在曲線或路徑的定義域上，以 `count` 個等距參數求值（含兩端，`count` ≥ 2），位於 `bezierkit.sampling`。

`Sample` 取樣的結果：唯讀的參數 `t` 與 `points`，提供座標陣列 `x`、`y`，並可逐一取出 `(t, point)`。

```
from bezierkit.sampling import UniformSampler

sample = UniformSampler(5).sample(curve)
print(sample.x)   # [0.      0.90625 2.      3.09375 4.     ]
```

11.2　四捨五入與座標變換

文字格式的匯出器以固定的小數位數 `precision` 寫出每個座標，並接受 `transform`：在寫出前把每個控制點（`Point`）對應到二維 `Point`，例如由資料座標轉為頁面座標。

命題 41 (四捨五入誤差) *設 B、$\tilde{B}$ 是同為 n 次的 $\mathbb{R}^d$ 中 Bézier 曲線，控制點分別為 P_i 與 $\tilde{P}_i$，且對每個 i 與每個座標 k 有 $|\tilde{P}_{i,k} - P_{i,k}| \le \varepsilon$。則對每個 $t \in [0,1]$ 與每個 k，*

$$|\tilde{B}_k(t) - B_k(t)| \le \varepsilon, \quad \left\|\tilde{B}(t) - B(t)\right\|_2 \le \sqrt{d}\,\varepsilon.$$

將每個座標四捨五入到小數點後 p 位時，$\varepsilon = \tfrac{1}{2}\cdot 10^{-p}$。

因此在預設的 `precision=6` 下，匯出的平面曲線處處與原曲線相差不超過 0.71×10^{-6}，不只在控制點上成立。依命題 8，仿射的 `transform` 是精確的；非仿射的變換能正確移動控制點，但一般不能正確移動控制點之間的曲線。

11.3　JSON

dumps　`dumps(⟨路徑⟩, *, metadata=None, indent=None)`
位於 `bezierkit.export.json`。以表 4 的版本化格式寫出路徑，鍵依字母排序，不含 NaN 或無窮大。

loads　`loads(⟨字串⟩)`
驗證文件並回傳 `PathDocument`，內含 `path` 與 `metadata`。格式或版本不明、線段不是恰好四個點、點的維度不符或數值非有限時，拋出 `ValueError`。

PathDocument　路徑與呼叫端的 metadata，即 `loads()` 的回傳值。

表 4
JSON 路徑格式第 1 版

鍵	值
`schema`	字串 `"bezierkit.path"`
`version`	整數 `1`；讀取端拒絕其他版本
`dimension`	每個點的維度 d
`subpaths`	非空陣列，元素為 `{"closed": bool, "segments": [...]}`，每個線段是四個含 d 個數值的點
`metadata`	JSON 物件，原樣傳遞、不解讀

```
from bezierkit.export.json import dumps, loads

text = dumps(path, metadata={"name": "arch"})
# {"dimension":2,"metadata":{"name":"arch"},"schema":"bezierkit.path",
#  "subpaths":[{"closed":false,"segments":[[[0.0,0.0],
[1.0,2.0],...]]}],"version":1}
assert loads(text).path.segments == path.segments
```

第 1 版已凍結：新增必要的鍵、改變封閉的語意或控制點的排列方式，都需要第 2 版。

11.4　SVG

to_svg_path_data　`to_svg_path_data(⟨路徑⟩, *, precision=6, transform=None)`

from_svg_path_data　`from_svg_path_data(⟨字串⟩)`
位於 `bezierkit.export.svg`。回傳平面路徑的路徑資料字串（即 `d` 屬性），只使用絕對座標的 `M`、`C` 與 `Z` 指令，每條子路徑一個 `M`；不產生 SVG 文件或樣式。`from_svg_path_data()` 解析同一子集；相對指令、直線與圓弧拋出 `ValueError`。

```
from bezierkit.export.svg import to_svg_path_data

print(to_svg_path_data(path, precision=2))   # M 0.00 0.00 C 1.00 2.00
3.00 2.00 4.00 0.00
```

11.5 TikZ

to_tikz `to_tikz(⟨路徑⟩, *, precision=6, transform=None, options=None)`

segment_to_tikz `segment_to_tikz(⟨segment⟩, *, precision=6, transform=None, include_start=True)`

位於 `bezierkit.export.tikz`。每條子路徑一個 `\draw` 指令，每個線段寫成 `.. controls (P1) and (P2) .. (P3)`，封閉子路徑以 `-- cycle` 結尾。`options` 原樣放入 `\draw[options]`；預設不加任何選項。`segment_to_tikz()` 只寫出一個線段、不含 `\draw`，供需要自行組合指令的呼叫端使用。

```
from bezierkit.export.tikz import to_tikz

print(to_tikz(path, precision=2, options="thick"))
# \draw[thick] (0.00,0.00) .. controls (1.00,2.00) and (3.00,2.00) ..
(4.00,0.00);
```

本手冊的每張圖都是這樣產生的：腳本以 bezierkit 建立曲線，用 `to_tikz()` 寫進 standalone LaTeX 文件，與座標軸、標籤放在一起，再編譯成 PDF。每張圖的 `.tex` 原始檔都隨手冊提供。

11.6 Matplotlib

from_path `from_path(⟨路徑⟩, *, transform=None)`

to_path `to_path(⟨路徑⟩)`

approximate_path `approximate_path(⟨路徑⟩, ⟨transform⟩, *, tolerance, max_depth=20)`

位於 `bezierkit.adapters.matplotlib`，需安裝 `matplotlib` 選用依賴。`from_path()` 精確轉換 Matplotlib `Path`：`MOVETO`、`LINETO`、`CURVE3`、`CURVE4` 與 `CLOSEPOLY` 都轉為三次曲線，直線與二次曲線依命題 22 升階。仿射的 `transform` 直接套用在控制點上，結果精確（詳見命題 8）；非仿射的變換拋出 `ValueError`。`to_path()` 反向轉換，每個線段一個 `CURVE4`。

非仿射變換（例如對數座標軸）不會把三次曲線對應到三次曲線。`approximate_path()` 是明確的選擇：它細分每個變換後的線段，直到中點與弦的距離在 `tolerance / 2` 以內，再以 `tolerance / 2` 對這些點呼叫 `fit_polyline`。它不宣稱結果精確。

第 12 節　命令列介面

安裝 `cli` 選用依賴後即可使用 `bezierkit` 指令，在命令列計算、取樣與建構曲線。預設輸出表格，也可輸出 JSON 或 CSV 給其他程式使用。

```
bezierkit evaluate --points "0,0" --points "1,2" --points "3,2" --
points "4,0" \
                   --t 0.5 --order 1 --json
# [{"t":0.5,"value":[4.5,0.0]}]
```

12.1　控制點

需要曲線的指令以重複的 `--points` 選項接收控制點，每個是以逗號分隔的座標（`"1,2"`，三維則為 `"1,2,3"`）。省略 `--points` 時，指令從標準輸入讀取含 `control_points` 陣列的 JSON 物件，也就是 `construct` 的輸出，因此指令之間可以用管線串接。

12.2　指令

`evaluate`　`bezierkit evaluate --t ⟨數值⟩ [--points ...] [--order ⟨整數⟩] [--json]`

在每個 `--t` 計算曲線值，或其 `--order` 階導數（預設 0）。每個值可為數字或含端點的範圍 `start:stop:step`，例如 `0:1:0.25`；`--t` 可重複。

`sample`　`bezierkit sample [--points ...] [--count ⟨整數⟩] [--format table|csv|json] [--output ⟨路徑⟩]`

以 `--count` 個（預設 50）等距參數取樣，含兩端，每列為 `t`、`x`、`y`（三維另有 `z`，更高維為 `c3`、`c4`……）。`--output` 寫入檔案而非標準輸出。

`construct slopes` / `construct tangents`　`bezierkit construct slopes --start ⟨point⟩ --end ⟨point⟩ --start-slope ⟨浮點數⟩ --end-slope ⟨浮點數⟩ [--start-handle --end-handle]`

`bezierkit construct tangents --start ... --end ... --start-direction ⟨vector⟩ --end-direction ⟨vector⟩ [...]`

以 `PlanarSlopes` 或 `TangentDirections`（詳見第 8 節）建構三次曲線，並以 JSON 印出控制點。

```
bezierkit construct slopes --start "0,5" --end "5,0" \
                           --start-slope -2 --end-slope -0.3 \
| bezierkit sample --count 3 --format csv
# t,x,y
# 0.0,0.0,5.0
# 0.5,2.3085202413545525,2.272345260462411
# 1.0,5.0,0.0
```

12.3　錯誤

無效輸入（例如格式錯誤的點，或超出 $[0,1]$ 的參數）會在標準錯誤輸出印出 `Error: ...`，結束狀態為 1，不顯示 traceback。在指令前加上全域選項 `--debug` 可改為顯示 Python traceback。

附錄 A　證明

本附錄依各章出現的順序，證明其中陳述的引理、命題、定理與推論。以下 $b_{i,n}$ 為定義 1 的 Bernstein 多項式，B 為定義 5 的 Bézier 曲線，次數為 n，控制點為 $P_0, \dots, P_n$。若某個結果在文獻中也有證明，章節中會標明出處；此處的證明寫成自足的形式，使手冊不依賴某一版本的書。

A.1　Bernstein 基底與求值

引理 2 的證明： 第一式。 依 j 的位置分別討論。

- 若 $j<0$ 或 $j>n$，三個係數都為零，因為 $j>n$ 蘊含 $j-1>n-1$。
- 若 $j=0$，左邊為 $1+0=1=\binom{n}{0}$。
- 若 $j=n$，左邊為 $0+1=1=\binom{n}{n}$。
- 若 $1\le j\le n-1$，通分為 $j!(n-j)!$：

$$\binom{n-1}{j}+\binom{n-1}{j-1}=\frac{(n-1)!(n-j)}{j!(n-j)!}+\frac{(n-1)!j}{j!(n-j)!}=(n-1)!\frac{n}{j!(n-j)!}=\binom{n}{j}.$$

第二式。 若 $i<1$ 或 $i>n$，兩邊都為零：$i=0$ 時左邊含因子 $i=0$，且 $\binom{n-1}{-1}=0$；$i<0$ 與 $i>n$ 時每個係數都為零。若 $1\le i\le n$，

$$i\binom{n}{i}=i\frac{n!}{i!(n-i)!}=n\frac{(n-1)!}{(i-1)!(n-i)!}=n\binom{n-1}{i-1}.$$

□

命題 3 的證明： 次數不超過 n 的實係數多項式空間 Π_n 的維度為 $n+1$，一組基底為 $1,t,\dots,t^n$。$n+1$ 個多項式 $b_{0,n},\dots,b_{n,n}$ 都屬於 Π_n，因此只需證明它們線性獨立。設對所有 t 有 $\sum_{i=0}^{n}c_ib_{i,n}(t)=0$。對 $t\in(0,1)$ 除以 $(1-t)^n>0$，並令 $u=t/(1-t)\in(0,\infty)$：

$$\sum_{i=0}^{n}c_i\binom{n}{i}u^i=0.$$

有無窮多個根的 u 的多項式必為零多項式，所以每個 i 都有 $c_i\binom{n}{i}=0$。因 $\binom{n}{i}\neq 0$，所有 $c_i=0$。 □

命題 4 的證明： 當 $t\in[0,1]$ 時，t 與 $1-t$ 都非負，因此每個 $b_{i,n}(t)$ 都非負。由二項式定理，

$$\sum_{i=0}^{n}\binom{n}{i}t^i(1-t)^{n-i}=(t+(1-t))^n=1.$$

□

命題 6 的證明： 約定 $0^0=1$。$b_{i,n}(0)=\binom{n}{i}0^i$ 在 $i=0$ 時為 1，在 $i\ge 1$ 時為 0。同理 $b_{i,n}(1)=\binom{n}{i}0^{n-i}$ 在 $i=n$ 時為 1，在 $i<n$ 時為 0。因此 $B(0)=\sum_i b_{i,n}(0)P_i=P_0$，$B(1)=\sum_i b_{i,n}(1)P_i=P_n$。 □

推論 7 的證明：由命題 4，$B(t)=\sum_i\lambda_iP_i$，其中 $\lambda_i=b_{i,n}(t)\ge 0$ 且 $\sum_i\lambda_i=1$。依定義，點的凸組合位於這些點的凸包內。 □

命題 8 的證明： 由 M 的線性與命題 4（$\sum_i b_{i,n}(t)=1$），

$$A(B(t)) = M\sum_i b_{i,n}(t)P_i + v$$

$$= \sum_i b_{i,n}(t)MP_i + \left(\sum_i b_{i,n}(t)\right)v$$

$$= \sum_i b_{i,n}(t)(MP_i + v) = \sum_i b_{i,n}(t)A(P_i).$$

□

定理 *10* 的證明：對 r 作歸納。$r = 0$ 時，$P_i^{(0)} = P_i = b_{0,0}(t)P_i$。假設對 $r-1$ 與每個容許的 i 都成立。對 $0 \le i \le n-r$，i 與 $i+1$ 對 $r-1$ 都是容許的，因此

$$P_i^{(r)} = (1-t)\sum_{j=0}^{r-1} b_{j,r-1}(t)P_{i+j} + t\sum_{j=0}^{r-1} b_{j,r-1}(t)P_{i+1+j}$$

$$= \sum_{j=0}^{r}\Big[(1-t)b_{j,r-1}(t) + tb_{j-1,r-1}(t)\Big]P_{i+j},$$

其中第二個和式以 $j+1 \to j$ 重新編號，兩端由約定 $b_{-1,r-1} = b_{r,r-1} = 0$ 涵蓋。對 $0 \le j \le r$，由引理 2，方括號為

$$\binom{r-1}{j}t^j(1-t)^{r-j} + \binom{r-1}{j-1}t^j(1-t)^{r-j} = \binom{r}{j}t^j(1-t)^{r-j} = b_{j,r}(t).$$

取 $r = n$、$i = 0$，結論即為 $P_0^{(n)} = B(t)$。 □

引理 *11* 的證明：把 $b_{i,n}(t)$ 乘以 $1 = (1-t) + t$：

$$b_{i,n}(t) = \binom{n}{i}t^i(1-t)^{n+1-i} + \binom{n}{i}t^{i+1}(1-t)^{n-i}.$$

對 $0 \le i \le n$，$\binom{n+1}{i} = (n+1)/(n+1-i)\cdot\binom{n}{i}$ 且 $\binom{n+1}{i+1} = (n+1)/(i+1)\cdot\binom{n}{i}$，所以

$$\binom{n}{i} = \frac{n+1-i}{n+1}\binom{n+1}{i} = \frac{i+1}{n+1}\binom{n+1}{i+1}.$$

因此第一項是 $b_{i,n+1}(t)$ 的 $(n+1-i)/(n+1)$ 倍，第二項是 $b_{i+1,n+1}(t)$ 的 $(i+1)/(n+1)$ 倍。 □

命題 *12* 的證明：由引理 11，

$$B(t) = \sum_{i=0}^{n}\left[\frac{n+1-i}{n+1}b_{i,n+1}(t) + \frac{i+1}{n+1}b_{i+1,n+1}(t)\right]P_i$$

$$= \sum_{k=0}^{n}\frac{n+1-k}{n+1}b_{k,n+1}(t)P_k + \sum_{k=1}^{n+1}\frac{k}{n+1}b_{k,n+1}(t)P_{k-1},$$

其中第一個和式取 $k = i$，第二個取 $k = i+1$。第一個和式的係數在 $k = n+1$ 為零，可延伸到 $k = n+1$；第二個和式同理可延伸到 $k = 0$。合併 $b_{k,n+1}(t)$ 的係數即得 Q_k。 □

A.2　導數、反轉與分割

引理 13 的證明： 依 i 的取值分別討論。

- 若 $i<0$ 或 $i>n$，則 $b_{i,n}=0$，右邊兩項也為零。
- 若 $i=0$，$b_{0,n}(t)=(1-t)^n$ 的導數為 $-n(1-t)^{n-1}=n\big[b_{-1,n-1}(t)-b_{0,n-1}(t)\big]$。
- 若 $i=n$，$b_{n,n}(t)=t^n$ 的導數為 $nt^{n-1}=n\big[b_{n-1,n-1}(t)-b_{n,n-1}(t)\big]$。
- 若 $1\le i\le n-1$，由乘積法則，

$$b'_{i,n}(t)=\binom{n}{i}\big[it^{i-1}(1-t)^{n-i}-(n-i)t^i(1-t)^{n-i-1}\big].$$

  由引理 2，$i\binom{n}{i}=n\binom{n-1}{i-1}$；以 $n-i$ 代替 i 並利用 $\binom{n}{n-i}=\binom{n}{i}$，得 $(n-i)\binom{n}{i}=n\binom{n-1}{i}$。因此兩項分別為 $nb_{i-1,n-1}(t)$ 與 $nb_{i,n-1}(t)$。

□

定理 14 的證明： 由線性與引理 13，

$$B'(t)=n\sum_{i=0}^{n}b_{i-1,n-1}(t)P_i-n\sum_{i=0}^{n}b_{i,n-1}(t)P_i.$$

第一個和式中 $i=0$ 的項為零；代入 $k=i-1$ 得 $\sum_{k=0}^{n-1}b_{k,n-1}(t)P_{k+1}$。第二個和式中 $i=n$ 的項為零。因此

$$B'(t)=n\sum_{k=0}^{n-1}b_{k,n-1}(t)(P_{k+1}-P_k).$$

□

推論 15 的證明： 由定理 14，B' 是控制點為 $D_k=n(P_{k+1}-P_k)$ $(k=0,\dots,n-1)$ 的 $n-1$ 次 Bézier 曲線。對它套用命題 6，得 $B'(0)=D_0$、$B'(1)=D_{n-1}$。 □

引理 16 的證明： 對 $0\le i\le n$，由 $\binom{n}{i}=\binom{n}{n-i}$，

$$b_{i,n}(1-t)=\binom{n}{i}(1-t)^it^{n-i}=\binom{n}{n-i}t^{n-i}(1-t)^i=b_{n-i,n}(t).$$

其他 i 兩邊都為零。 □

命題 17 的證明： 令 $j=n-i$ 重新編號，並利用引理 16：

$$\sum_{i=0}^{n}b_{i,n}(t)P_{n-i}=\sum_{j=0}^{n}b_{n-j,n}(t)P_j=\sum_{j=0}^{n}b_{j,n}(1-t)P_j=B(1-t).$$

□

定理 18 的證明： *左段。* 因 $1-cs=(1-s)+s(1-c)$，由二項式定理，

$$B(cs) = \sum_{i=0}^{n} \binom{n}{i} (cs)^i [(1-s) + s(1-c)]^{n-i} P_i$$

$$= \sum_{i=0}^{n} \sum_{k=0}^{n-i} \binom{n}{i} \binom{n-i}{k} c^i (1-c)^k s^{i+k} (1-s)^{n-i-k} P_i.$$

代入 $j = i + k$，則 $0 \le i \le j \le n$ 且 $n - i - k = n - j$。$\binom{n}{i}\binom{n-i}{j-i}$ 與 $\binom{n}{j}\binom{j}{i}$ 都等於 $n!/(i!(j-i)!(n-j)!)$，故

$$B(cs) = \sum_{j=0}^{n} \binom{n}{j} s^j (1-s)^{n-j} \sum_{i=0}^{j} \binom{j}{i} c^i (1-c)^{j-i} P_i$$

$$= \sum_{j=0}^{n} b_{j,n}(s) P_0^{(j)}.$$

最後一步用定理 10 取 $r = j$、$i = 0$：$P_0^{(j)} = \sum_{i=0}^{j} b_{i,j}(c) P_i$。這就是 L_j 的結論。

右段。 令 $\overline{B}$ 為控制點 $\overline{P}_i = P_{n-i}$ 的曲線，由命題 17，$\overline{B}(u) = B(1-u)$；令 $\overline{c} = 1 - c$。對 $\overline{B}$ 套用左段，

$$\overline{B}(\overline{c}s) = \sum_{j=0}^{n} b_{j,n}(s) \overline{L}_j,$$

其中由引理 16 與代換 $l = j - i$，

$$\overline{L}_j = \sum_{i=0}^{j} b_{i,j}(1-c) P_{n-i} = \sum_{l=0}^{j} b_{l,j}(c) P_{n-j+l} = P_{n-j}^{(j)} = R_{n-j}.$$

倒數第二個等號是定理 10 取 $r = j$、$i = n - j$，此指標是容許的，因為 $n - j \le n - r$。因此再用一次引理 16，

$$B(c + (1-c)s) = \overline{B}(\overline{c}(1-s)) = \sum_{j=0}^{n} b_{j,n}(1-s) R_{n-j}$$

$$= \sum_{j=0}^{n} b_{n-j,n}(s) R_{n-j} = \sum_{k=0}^{n} b_{k,n}(s) R_k.$$

□

推論 19 的證明： 令 $a = t_0$、$e = t_1$，則 $0 \le a < e \le 1$。由定理 18 取 $c = e$，左段是控制點為 $L_0, \ldots, L_n$ 的 Bézier 曲線 $u \mapsto B(eu)$。對它再套用定理 18，取 $c' = a/e \in [0, 1)$ 並保留右段：

$$s \mapsto B\Big(e\Big(\frac{a}{e} + \Big(1 - \frac{a}{e}\Big)s\Big)\Big) = B(a + (e-a)s).$$

□

A.3　三次線段與路徑

命題 21 的證明： 展開三次 Bernstein 形式並依 t 的次方合併，

$$\begin{aligned} B(t) &= (1-t)^3P_0 + 3t(1-t)^2P_1 + 3t^2(1-t)P_2 + t^3P_3 \\ &= \big(1 - 3t + 3t^2 - t^3\big)P_0 + \big(3t - 6t^2 + 3t^3\big)P_1 + \big(3t^2 - 3t^3\big)P_2 + t^3P_3 \\ &= at^3 + bt^2 + ct + P_0, \end{aligned}$$

其中 a、b、c 如命題所述。

固定座標 k。B_k 是多項式，在緊緻區間 $[0,1]$ 上連續，因此在其上取得最小值與最大值。設 t^* 是取得其中一個的點。若 $t^* \in (0,1)$，則 B_k 在內點有極值且可微，故 $B_k'(t^*) = 0$（Fermat 定理）。分兩種情形。

- B_k' 不恆為零。它是次數至多為 2 的多項式，至多有兩個根；若 t^* 在 $(0,1)$ 內，它就是其中之一。每種情形都有 $t^* \in T_k$。
- B_k' 恆為零。則 B_k 為常數，其極值在 $t = 0$ 也取得，且 $0 \in T_k$。

因此 B_k 在 $[0,1]$ 上的最小值與最大值，等於它在有限集 $T_k \subset [0,1]$ 上的最小值與最大值，也就是命題中的 α_k 與 β_k，因為它們就是 B_k 在曲線上的下確界與上確界。由定義 20，邊界框是各區間 $[\alpha_k, \beta_k]$ 的乘積：包含曲線的方框必包含每個值 $B_k(t)$，因而包含 α_k 與 β_k；而這些區間的乘積本身就包含曲線。 □

命題 *22* 的證明：直線。直線 $t \mapsto (1-t)P_0 + tP_3$ 是控制點為 $R_0 = P_0$、$R_1 = P_3$ 的一次 Bézier 曲線。由命題 12 取 $n = 1$，它是控制點為

$$Q_0 = P_0, \quad Q_1 = \frac{1}{2}P_0 + \frac{1}{2}P_3, \quad Q_2 = P_3$$

的二次曲線。再套用命題 12 取 $n = 2$，得到控制點為

$$\begin{aligned} S_0 &= Q_0 = P_0, \\ S_1 &= \frac{1}{3}Q_0 + \frac{2}{3}Q_1 = \frac{2}{3}P_0 + \frac{1}{3}P_3, \\ S_2 &= \frac{2}{3}Q_1 + \frac{1}{3}Q_2 = \frac{1}{3}P_0 + \frac{2}{3}P_3, \\ S_3 &= Q_2 = P_3 \end{aligned}$$

的三次曲線，亦即對 $i = 0, \ldots, 3$ 有 $S_i = P_0 + \frac{i}{3}(P_3 - P_0)$。

二次曲線。由命題 12 取 $n = 2$ 與控制點 P_0, C, P_3，三次控制點為

$$Q_0 = P_0, \quad Q_1 = \frac{1}{3}P_0 + \frac{2}{3}C, \quad Q_2 = \frac{2}{3}C + \frac{1}{3}P_3, \quad Q_3 = P_3,$$

即 $Q_1 = P_0 + \frac{2}{3}(C - P_0)$、$Q_2 = P_3 + \frac{2}{3}(C - P_3)$。

每一步都是關於 t 的恆等式，因此曲線與其參數化不變。 □

命題 *25* 的證明：令 B_S、B_T 為控制點分別是 P_i 與 Q_i 的三次 Bézier 曲線，使 $S(u) = B_S((u - u_0 + h_S)/h_S)$、$T(u) = B_T((u - u_0)/h_T)$。在 $u = u_0$ 處，兩個引數分別為 1 與 0。由命題 6，$S(u_0) = P_3$、$T(u_0) = Q_0$。由連鎖律與推論 15，

$$S'(u_0)=\frac{B_S'(1)}{h_S}=\frac{3(P_3-P_2)}{h_S},\quad T'(u_0)=\frac{B_T'(0)}{h_T}=\frac{3(Q_1-Q_0)}{h_T}.$$

1. 由定義 23 取 $k=0$，C 在 u_0 為 C^0 若且唯若 $S(u_0)=T(u_0)$，即 $P_3=Q_0$。
2. 當 $P_3=Q_0$，定義 23 取 $k=1$ 多出 $S'(u_0)=T'(u_0)$ 的條件，兩邊除以 3 即所述等式。
3. 當 $P_3=Q_0$，由定義 24，接點為 G^1 若且唯若 $S'(u_0)\neq 0$、$T'(u_0)\neq 0$，且存在 $\lambda>0$ 使 $T'(u_0)=\lambda S'(u_0)$。兩個切向量皆非零，若且唯若 $P_3\neq P_2$ 且 $Q_1\neq Q_0$，此時

$$Q_1-Q_0=\left(\lambda\frac{h_T}{h_S}\right)(P_3-P_2).$$

   反之，若 $Q_1-Q_0=\mu(P_3-P_2)$ 且 $\mu>0$，取 $\lambda=\mu h_S/h_T>0$，即得 $T'(u_0)=\lambda S'(u_0)$。
4. 均勻參數化下，m 段路徑的每段占 $1/m$，故 $h_S=h_T$，第 2 項的等式化為 $P_3-P_2=Q_1-Q_0$。

□

A.4　建構與插值

命題 26 的證明：對具所述控制點的曲線 B，由命題 6 得 $B(0)=P_0$、$B(1)=P_3$，由推論 15 得

$$B'(0)=3(P_1-P_0)=D_0,\quad B'(1)=3(P_3-P_2)=D_1.$$

再設 Γ 是任一次數至多為 3、滿足 $\Gamma(0)=P_0$、$\Gamma(1)=P_3$、$\Gamma'(0)=D_0$、$\Gamma'(1)=D_1$ 的多項式曲線。對每個座標套用命題 3，存在唯一的 $R_0,\ldots,R_3$ 使 $\Gamma(t)=\sum_i b_{i,3}(t)R_i$。對控制點 R_i 作上述計算，得

$$\Gamma(0)=R_0,\quad \Gamma(1)=R_3,\quad \Gamma'(0)=3(R_1-R_0),\quad \Gamma'(1)=3(R_3-R_2).$$

與四個條件比較，得 $R_0=P_0$、$R_3=P_3$、$R_1=P_0+D_0/3$、$R_2=P_3-D_1/3$。所以 R_i 就是所述的控制點，$\Gamma=B$。□

命題 28 的證明：令 $G(s)=H(x_0+hs)$。映射 $H\mapsto G$ 是次數至多為 3 的多項式集合上的雙射，反映射為 $G\mapsto G((x-x_0)/h)$。由連鎖律，$H(x_i)=f_i$ 與 $H'(x_i)=m_i$（$i=0,1$）等價於

$$G(0)=f_0,\quad G(1)=f_1,\quad G'(0)=hm_0,\quad G'(1)=hm_1.$$

由命題 26 取 $d=1$、$P_0=f_0$、$P_3=f_1$、$D_0=hm_0$、$D_1=hm_1$，恰有一個次數至多為 3 的多項式 G 滿足這四個條件，其 Bernstein 係數為 $c_0,\ldots,c_3$。因此 H 存在、唯一，且等於 $\sum_i b_{i,3}((x-x_0)/h)c_i$。□

命題 29 的證明：在 $t=t_0$ 與 $t=t_1$，線段分別於 $s=0$ 與 $s=1$ 求值，由命題 6 得 $H(t_0)=P_0$、$H(t_1)=P_3$。由連鎖律，$H'(t)=B'(s)/h$。由推論 15，$B'(0)=3(P_1-P_0)=hD_0$、$B'(1)=3(P_3-P_2)=hD_1$，故 $H'(t_0)=D_0$、$H'(t_1)=D_1$。證明中不需要 $h>0$。□

命題 *30* 的證明： 將線段寫成 $(X(s), Y(s))$。X 的控制點為

$$x_0,\quad x_0+\frac{h}{3},\quad x_1-\frac{h}{3}=x_0+2\frac{h}{3},\quad x_1,$$

由命題 22（由 x_0 到 x_1 的直線），它們正是直線 $s\mapsto x_0+hs$ 的控制點，故 $X(s)=x_0+hs$。Y 的控制點為 $y_0, y_0+hm_0/3, y_1-hm_1/3, y_1$，即命題 28 中對資料 $y_i=f(x_i)$、$m_i=f'(x_i)$ 的係數 $c_0,\dots,c_3$。因此 $Y(s)=H(x_0+hs)$，線段為 $s\mapsto(x_0+hs, H(x_0+hs))$。□

定理 *31* 的證明： $x\in\{x_0,x_1\}$ 的情形是平凡的：第一個公式兩邊皆為零，對任何 ξ 都成立。設 $x_0<x<x_1$，並令

$$E=f-H,\quad w(t)=(t-x_0)^2(t-x_1)^2,\quad K=\frac{E(x)}{w(x)},\quad g=E-Kw.$$

則 $w(x)>0$，g 四階連續可微，且 $g(x_0)=g(x)=g(x_1)=0$：E 與 w 在 x_0、x_1 為零，而 $g(x)=E(x)-Kw(x)=0$。此外 $g'(x_0)=g'(x_1)=0$：由定義 27，$E'(x_i)=f'(x_i)-H'(x_i)=0$，而 w 在每個 x_i 有二重根，故 $w'(x_i)=0$。

由 Rolle 定理（Burden & Faires (2011) 的定理 1.7），g' 在某個 $\xi_1\in(x_0,x)$ 與某個 $\xi_2\in(x,x_1)$ 為零。因此 g' 有四個相異零點 $x_0<\xi_1<\xi_2<x_1$。g' 連續且三階可微，由廣義 Rolle 定理（Burden & Faires (2011) 的定理 1.10），存在 $\xi\in(x_0,x_1)$ 使 $g^{(4)}(\xi)=0$。由於 H 的次數至多為 3，而 w 是首項係數為 1 的四次多項式，$H^{(4)}=0$、$w^{(4)}=24$，因此

$$0=g^{(4)}(\xi)=f^{(4)}(\xi)-24K,\quad \text{即}\quad K=\frac{f^{(4)}(\xi)}{24}.$$

故 $f(x)-H(x)=E(x)=Kw(x)$，這就是第一個公式。

關於界限：$(x-x_0)(x_1-x)$ 是兩個和為 h 的非負數之積，至多為 $(h/2)^2$。因此 $w(x)\le h^4/16$，

$$|f(x)-H(x)|\le\frac{M}{24}\cdot\frac{h^4}{16}=\frac{Mh^4}{384}.$$

□

A.5　擬合與等值線

推論 *32* 的證明： 設 $[a,b]$ 是深度 k 的區間，則 $b-a=L/2^k$。在此區間上，Hermite 線段的每個座標都是 C 同一座標的三次 Hermite 插值多項式（詳見命題 29 與命題 28），因此由定理 31，其誤差不超過 $M(b-a)^4/384$。任一參數處的歐氏誤差至多為最大座標誤差的 $\sqrt{d}$ 倍，因為對 $v\in\mathbb{R}^d$ 有 $\|v\|_2\le\sqrt{d}\|v\|_\infty$：事實上 $\|v\|_2^2=\sum_k v_k^2\le d\|v\|_\infty^2$。特別地，在量測參數上取最大值得到的實測誤差滿足

$$e\le\sqrt{d}M\frac{L^4}{384\cdot 2^{4k}}.$$

此值不超過 ε，若且唯若 $2^{4k} \geq \sqrt{d}ML^4/(384\varepsilon)$，也就是 $k \geq \frac{1}{4}\log_2\!\left(\sqrt{d}ML^4/(384\varepsilon)\right)$。每個 $k \geq k^*$ 都滿足此式，所以該深度的每個區間都會被接受。

再設 `max_depth` $\geq k^*$ 且 `max_segments` $\geq 2^{k^*}$。區間只有在不合格時才會被分割，而這只發生在深度 $k < k^*$。因此所有區間的深度至多為 k^*，不會因深度而拋出 `ToleranceNotMet`。被接受的區間是把 $[t_0, t_1]$ 對半至多 k^* 次得到的互不重疊的片段，所以至多有 2^{k^*} 個。不合格的區間至少會產生兩個被接受的線段，因此「目前已接受的線段數加二」不會超過這個最終數目，而它至多為 `max_segments`；所以線段數上限也不會被超過。 □

命題 34 的證明： 以 $\mathrm{rdp}(i, j)$ 表示演算法對指標 $i < j$ 執行的遞迴步驟，它回傳由 i 到 j 的遞增指標列表。對 $j - i$ 作強歸納，證明對回傳列表中任兩個相鄰指標 $u < v$，每個滿足 $u \leq l \leq v$ 的指標 l 都有 $\mathrm{dist}(v_l, [v_u, v_v]) \leq \varepsilon$。

*基底。*若 $j \leq i + 1$，列表為 (i, j)，且介於 i 與 j（含）之間的指標只有 i 與 j，其頂點就是弦的端點，距離為零。

*歸納步驟。*否則令 δ^* 為嚴格介於 i 與 j 之間的頂點到 $\left[v_i, v_j\right]$ 的最大距離。若 $\delta^* \leq \varepsilon$，列表為 (i, j)，由 δ^* 的選取以及端點距離為零，結論成立。若 $\delta^* > \varepsilon$，步驟選取指標 m，$i < m < j$，並回傳 $\mathrm{rdp}(i, m)$ 的列表接上 $\mathrm{rdp}(m, j)$ 的列表，m 只出現一次。結果中任兩個相鄰指標，在這兩個列表之一中也是相鄰的。由於 $m - i$ 與 $j - m$ 都小於 $j - i$，歸納假設適用於兩者。

步驟 2 與 3 對步驟 2 保留的每兩個相鄰頂點執行 `rdp` 並串接結果。輸出恰保留回傳的指標 $a_0 < \ldots < a_m$，其中每兩個相鄰指標在某個列表中也相鄰。這就證明了不等式。輸出線段的 `fit_error` 是 $\max_{a_j \leq i \leq a_{j+1}} \mathrm{dist}\!\left(v_i, \left[v_{a_j}, v_{a_{j+1}}\right]\right)$，因此不超過 ε。 □

推論 35 的證明：

1. 到凸集的距離是凸函數。對線段 $K = [a, b]$，設 x_1, x_2 滿足 $\mathrm{dist}(x_i, K) \leq \varepsilon$，並在 $y_i \in K$ 取得（由緊緻性，最小值存在），$\lambda \in [0, 1]$。則 $\lambda y_1 + (1 - \lambda) y_2 \in K$，由三角不等式，

   $$\|\lambda x_1 + (1-\lambda)x_2 - (\lambda y_1 + (1-\lambda)y_2)\|_2 \leq \lambda\|x_1 - y_1\|_2 + (1-\lambda)\|x_2 - y_2\|_2 \leq \varepsilon.$$

   所以 $\mathrm{dist}(\lambda x_1 + (1-\lambda)x_2, K) \leq \varepsilon$。

   折線上的點 x 位於某條邊 $[v_i, v_{i+1}]$ 上，即 $x = \lambda v_i + (1-\lambda)v_{i+1}$。選取 j 使 $a_j \leq i < a_{j+1}$；因指標為整數，$i + 1 \leq a_{j+1}$，由命題 34，v_i 與 v_{i+1} 都在 $\left[v_{a_j}, v_{a_{j+1}}\right]$ 的 ε 範圍內。由剛證的凸性，x 亦然。

2. 通過步驟 1 的點就是某個 v_i，由命題 34，它離某條弦不超過 ε。被刪除的輸入點 p 與其前一個保留的頂點（記為 v）相距不超過 δ；設弦 $[a, b]$ 與 v 相距不超過 ε，則

   $$\mathrm{dist}(p, [a, b]) \leq \|p - v\|_2 + \mathrm{dist}(v, [a, b]) \leq \delta + \varepsilon.$$

封閉折線結尾重複的起點在同樣條件下被刪除，取 $v = v_0$ 即可。 □

命題 *40* 的證明：把 $[0,1]^2$ 的四個角代入 u，每個角只有一項不為零且係數為 1，所以 u 在角點取角點值。在 $s=t=1/2$，四個權重乘積都等於 $\frac{1}{2}\cdot\frac{1}{2}=\frac{1}{4}$，因此 $u\left(\frac{1}{2},\frac{1}{2}\right)=\frac{1}{4}(f_{00}+f_{10}+f_{11}+f_{01})$。 □

定理 *38* 的證明： 因為 $\nabla F(p)\neq 0$，$F_y(p)\neq 0$ 或 $F_x(p)\neq 0$。

情形 $F_y(p)\neq 0$。套用定理 37：存在 I、J 與 C^1 函數 $\varphi: I\to J$，$\varphi(p_x)=p_y$，且 $L_c\cap(I\times J)=\{(x,\varphi(x)):x\in I\}$。令 $N=I\times J$、$\gamma(x)=(x,\varphi(x))$。對 $F(x,\varphi(x))=c$ 用連鎖律微分，在 $\gamma(x)$ 處得 $F_x+F_y\varphi'=0$。由 F_y 的連續性，可縮小 I 與 J 使 $F_y\neq 0$ 在 N 上成立；於是

$$\gamma'(x)=(1,\varphi'(x))=\left(1,-\frac{F_x}{F_y}\right)=\frac{1}{F_y}\big(F_y,-F_x\big),$$

它非零，且在 $\gamma(x)$ 處平行於 $\big(F_y,-F_x\big)$。

情形 $F_x(p)\neq 0$。交換 x 與 y 的角色：存在 C^1 函數 ψ 使 $L_c\cap N=\{(\psi(y),y)\}$，曲線 $\gamma(y)=(\psi(y),y)$，且 $\psi'=-F_y/F_x$，故

$$\gamma'(y)=\left(-\frac{F_y}{F_x},1\right)=-\frac{1}{F_x}\big(F_y,-F_x\big),$$

同樣非零且平行於 $\big(F_y,-F_x\big)$。

最後 $\nabla F\cdot\big(F_y,-F_x\big)=F_xF_y-F_yF_x=0$。 □

A.6　匯出

命題 *41* 的證明： 兩條曲線的差為

$$\tilde{B}(t)-B(t)=\sum_{i=0}^{n}b_{i,n}(t)\big(\tilde{P}_i-P_i\big).$$

由命題 4，權重 $b_{i,n}(t)$ 非負且和為 1，因此對每個座標，

$$|\tilde{B}_k(t)-B_k(t)|\le\sum_{i=0}^{n}b_{i,n}(t)\,|\tilde{P}_{i,k}-P_{i,k}|\le\varepsilon.$$

由於對 $v\in\mathbb{R}^d$ 有 $\|v\|_2\le\sqrt{d}\|v\|_\infty$（見 Golub & Van Loan (2013) 第 2.2 節），得 $\big\|\tilde{B}(t)-B(t)\big\|_2\le\sqrt{d}\varepsilon$。

匯出器以 Python 的定點格式寫出每個座標的 p 位小數，得到最接近該座標的 10^{-p} 的整數倍，與原座標相差至多 $\frac{1}{2}\cdot 10^{-p}$。匯出器也把絕對值小於 $\frac{1}{2}\cdot 10^{-p}$ 的座標寫成 0，改變量即該絕對值，同樣至多為 $\frac{1}{2}\cdot 10^{-p}$。 □

更新紀錄

本章列出影響套件使用的版本變更，不含僅修改文件的版本；各條目依 l3doc 慣例標註於它所描述的功能旁，頁碼即該功能的實際頁碼。

v1.0.0 (2026-10-04)

一般：第一個正式版；公開 API 依語意化版本管理 ····· 2

`bezierkit`：持續整合測試 Python 3.10、3.11、3.12 與 3.13 ····· 3

`PiecewiseBezier.segment`：`t0` 落在線段內部時回傳正確的區間；先前會先在 `t0` 分割再對後半段重新參數化，導致終點錯誤 ····· 14

v0.5.0rc1 (2026-09-28)

一般：首次發布到 PyPI：任意次數的 Bézier 曲線、三次線段與路徑、建構、Hermite 插值、擬合、等值線描繪、取樣、匯出器與命令列介面 ····· 2

指令索引

A

approximate_path 26

B

BernsteinBasis 7

BernsteinEvaluator 9

BezierCurve 7

BezierCurve.derivative 10

BezierCurve.reversed 10

BezierCurve.segment 11

BezierCurve.split 11

C

construct slopes 27

construct tangents 27

ContourSet 24

CubicBezierSegment 11

CubicBezierSegment.bounding_box 12

D

DeCasteljauEvaluator 8

dumps 25

E

EndpointDerivatives 15

evaluate 27

F

fit_graph 19

fit_parametric 18

fit_polyline 20

from_path 26

from_svg_path_data 25

G

graph_hermite 17

H

hermite_error_bound 18

I

Interval 5

L

LevelContours 24

line_to_cubic 13

loads 25

M

maximum_polyline_deviation 20

P

ParameterValues 5

parametric_hermite 16

PathDocument 25

PiecewiseBezier 13

PlanarSlopes 16

Point 5

PointSet 5

Q

quadratic_to_cubic 13

S

Sample 24

sample 27

segment_to_tikz 26

T

TangentDirections 15

to_cubic 13

to_path 26

to_svg_path_data 25

to_tikz 26

trace_implicit 22

U

UniformSampler 24

V

Vector 5

參考文獻

Apostol, T. M. (1974). *Mathematical Analysis* (2nd ed.). Addison-Wesley.

Athawale, T., & Johnson, C. R. (2019). Probabilistic Asymptotic Decider for Topological Ambiguity Resolution in Level-Set Extraction for Uncertain 2D Data. *IEEE Transactions on Visualization and Computer Graphics*, *25*(1), 1163–1172. https://doi.org/10.1109/TVCG.2018.2864505

Barsky, B. A., & DeRose, T. D. (1989). Geometric Continuity of Parametric Curves: Three Equivalent Characterizations. *IEEE Computer Graphics and Applications*, *9*(6), 60–69. https://doi.org/10.1109/38.41470

Bernstein, S. N. (1912). Démonstration du théorème de Weierstrass fondée sur le calcul des probabilités. *Communications De La Société Mathématique De Kharkov*, *13*, 1–2.

Bézier, P. (1966). Définition numérique des courbes et surfaces I. *Automatisme*, *11*, 625–632.

Burden, R. L., & Faires, J. D. (2011). *Numerical Analysis* (9th ed.). Brooks/Cole.

Douglas, D. H., & Peucker, T. K. (1973). Algorithms for the Reduction of the Number of Points Required to Represent a Digitized Line or Its Caricature. *Cartographica*, *10*(2), 112–122. https://doi.org/10.3138/FM57-6770-U75U-7727

Farin, G. (2002). *Curves and Surfaces for CAGD: A Practical Guide* (5th ed.). Morgan Kaufmann.

Farouki, R. T. (2012). The Bernstein Polynomial Basis: A Centennial Retrospective. *Computer Aided Geometric Design*, *29*(6), 379–419. https://doi.org/10.1016/j.cagd.2012.03.001

Floater, M. S. (2025,). *An Introduction to Spline Theory*. https://www.uio.no/studier/emner/matnat/math/MAT4170/v25/undervisningsmateriale/spline_notes.pdf

Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.

Hershberger, J., & Snoeyink, J. (1994). An O(n \log n) Implementation of the Douglas–Peucker Algorithm for Line Simplification. *Proceedings of the Tenth Annual Symposium on Computational Geometry*, 383–384. https://doi.org/10.1145/177424.178097

Lorensen, W. E., & Cline, H. E. (1987). Marching Cubes: A High Resolution 3D Surface Construction Algorithm. *ACM SIGGRAPH Computer Graphics*, *21*(4), 163–169. https://doi.org/10.1145/37402.37422

Müller, A. (2024). A Tour d'Horizon of de Casteljau's Work. *Computer Aided Geometric Design*, *113*, 102366. https://doi.org/10.1016/j.cagd.2024.102366

Munkres, J. R. (1991). *Analysis on Manifolds*. Addison-Wesley.

Nielson, G. M., & Hamann, B. (1991). The Asymptotic Decider: Resolving the Ambiguity in Marching Cubes. *Proceedings of Visualization '91*, 83–91. https://doi.org/10.1109/VISUAL.1991.175782

Prautzsch, H., Boehm, W., & Paluszny, M. (2002). *Bézier and B-Spline Techniques*. Springer. https://doi.org/10.1007/978-3-662-04919-8

Ramer, U. (1972). An Iterative Procedure for the Polygonal Approximation of Plane Curves. *Computer Graphics and Image Processing*, *1*(3), 244–256. https://doi.org/10.1016/S0146-664X(72)80017-0

Rudin, W. (1976). *Principles of Mathematical Analysis* (3rd ed.). McGraw-Hill.

Stoer, J., & Bulirsch, R. (2002). *Introduction to Numerical Analysis* (3rd ed.). Springer. https://doi.org/10.1007/978-0-387-21738-3